

Proof-Producing Automation for Lean 4

Integrating Leant/Djex, Z3, and Lean's native solver infrastructure

Central conclusion

The most promising architecture is not to trust an external solver and not to replace Lean's tactics. It is a proof-producing portfolio: Leant/Djex proposes typed structural proof skeletons; Z3 supplies semantic countermodels, unsat cores, quantifier instances, and synthesized parameters; Lean reconstructs a proof from those hints or checks a small certificate; and the kernel remains the sole acceptance boundary.

Prepared for:	Vladimir Reshetnikov
Date:	10 August 2026
Primary repositories:	leanprover/lean4 Z3Prover/z3 VladimirReshetnikov/Leant
Related pinned component:	VladimirReshetnikov/Djex

The analysis is pinned to exact commits in Chapter 2. The report is an architectural and implementation feasibility study; it does not claim that the proposed tactics or protocols have already been implemented.

Status, 14 August 2026: on the Leant/Djex side, the countermodel ingredient has since shipped as the finite-list-spine Length contract dialect with live Z3 counterexample ranking; solver output remains untrusted and candidates are reordered, never pruned. The Lean-side proof-producing portfolio proposed in this report remains future work.

Abstract

Lean 4 already contains the ingredients of a modern proof-producing automation platform: an extensible elaborator and metaprogramming layer; a backtrackable, proof-reconstructing congruence and E-matching engine in grind; replayable editor suggestions through TryThis; a pluggable library-premise selector; specialized arithmetic solvers; and a reflected bit-vector tactic whose external SAT result is accepted only after an LRAT certificate is checked in Lean. Recent work has also begun to let tactics share grind’s normalized state instead of operating as isolated black boxes.

Leant and its pinned Djex engine contribute a complementary capability. They search for Curry–Howard inhabitants rather than merely deciding a first-order formula. Djinn performs bounded but complete structural proof search on its admitted intuitionistic fragment and can distinguish a genuine refutation from bounded failure. Exference performs ranked heuristic synthesis. Leant serializes a substantial fragment of an elaborated Lean target, discovers bounded exact provider metadata from the active environment, renders candidates, and then re-elaborates every candidate as a temporary Lean example. This is already a strong untrusted- generator/trusted-checker arrangement. Its limitations are mainly representational: the current public result is rendered source text rather than a typed proof graph, proof search is exposed through a separate REPL rather than a native tactic, and a successful candidate is generally all-or-nothing rather than a skeleton with typed residual goals.

Z3 contributes strong semantic search over quantifier-free and quantified first-order theories, models, assumption-based unsat cores, optimization, fixedpoint/Horn solving, model-based projection, and configurable proof logging. Its outputs have different logical strengths. A model after sat can support a counterexample only when the encoding and decoder are validated. An unsat core is a premise-selection hint, not a proof. A raw Z3 proof object or self-validated inference log is not automatically an independently checked Lean certificate. A result of unknown must remain explicit; a model returned after unknown is not, in general, a valid counterexample. These distinctions should be represented in the API rather than buried in diagnostic strings.

The recommended system is a layered portfolio. A Lean-side goal snapshot and projection layer assigns stable identities to binders, constants, theorem premises, and source occurrences. Untrusted workers then contribute different artifacts: Leant/Djex supplies typed proof graphs and hole-bearing structural skeletons; Z3 supplies validated finite models, unsat cores, candidate instantiations, arithmetic coefficients, invariants, and branch-feasibility results; Lean-native tactics supply proof-producing normalization and decision procedures. A reconstruction layer turns all proof-bearing artifacts into Lean expressions or tactic scripts and leaves every unsupported step as an ordinary residual goal. Only kernel-accepted terms close a goal. Diagnostic artifacts are shown with weaker result labels and never silently promoted to proofs.

The report proposes concrete Lean and wire-level data structures, a native leant?/leant tactic, a typed proof-skeleton protocol, a staged z3? oracle, theory-specific reconstruction for EUF, linear arithmetic, bit vectors, arrays, and algebraic datatypes, cooperation with grind and LibrarySuggestions, counterexample-guided synthesis, induction and invariant synthesis, proof repair, deterministic scheduling, caching, process isolation, and a benchmark methodology. The highest-value first implementation is deliberately modest: expose Leant as a native suggestion tactic, retain current Lean re-elaboration as the acceptance boundary, and add typed holes before attempting a generic Z3 proof-object checker.

Contents

Abstract	i
1 Executive conclusions	1
1.1 Bottom line	1
1.2 The seven most valuable directions	1
1.3 What should not be built first	2
1.4 A concrete first result	3
2 Scope, method, and repository snapshots	4
2.1 Exact revisions	4
2.2 Questions investigated	4
2.3 Terminology	5
3 Lean 4 as the proof-producing host	6
3.1 The relevant architecture is already present	6
3.2 grind: a natural cooperation point	6
3.3 TryThis: the right user-interface seam	7
3.4 LibrarySuggestions: premise selection as a first-class service	7
3.5 Reflected checking in <code>bv_decide</code>	8
3.6 Implications for an external automation design	8
4 Leant and Djex as structural proof synthesizers	9
4.1 Current end-to-end path	9
4.2 The admitted structural fragment	9
4.3 Evidence and progress are already separated	10
4.4 Current finite bounds are part of the contract	10
4.5 Djinn and Exference form a useful internal portfolio	11
4.6 Where the current representation blocks deeper cooperation	11
4.7 A native tactic need not move Djex into Lean	12
5 Z3 as a semantic search and diagnostic engine	13
5.1 The useful outputs have different logical strengths	13
5.2 Z3's strongest early roles	14

5.2.1	Validated counterexample generation	14
5.2.2	Assumption-based unsat cores	14
5.2.3	Semantic branch pruning	14
5.2.4	Template solving and optimization	14
5.3	Proof logging is useful but not yet the trust boundary	15
5.4	Recent Z3 history reinforces the fail-closed design	15
5.5	Candidate theory frontier	15
6	Trust model and result-strength lattice	17
6.1	The trusted computing base	17
6.2	A result lattice, not a Boolean	17
6.3	Formal direction of projection guarantees	18
6.4	Projection reports	18
6.5	Protocol and parser trust	19
7	Unified proof-automation architecture	20
7.1	Overview	20
7.2	Goal snapshots	20
7.3	A common candidate and evidence broker	21
7.4	Typed proof graphs	21
7.5	Residual obligations are a feature	22
7.6	Capability-directed hole routing	22
7.7	Proof plans and diagnostics share source identities	23
8	A native Leant tactic	24
8.1	Why this is the highest-leverage first step	24
8.2	Proposed commands	24
8.3	First implementation: preserve the existing textual boundary	24
8.4	Second implementation: return structured candidates	25
8.5	Worker protocol	25
8.6	Provider and premise selection	26
8.6.1	Ranked seed inventory	26
8.6.2	Demand-driven expansion	26
8.7	Verification in the exact tactic state	26
8.8	Compatibility strategy	27
9	Typed, hole-bearing proof synthesis	28

9.1	From complete terms to proof skeletons	28
9.2	Hole creation rules	28
9.3	Skeleton search objectives	28
9.4	Cooperative completion	29
9.5	Shared subgoals and DAG search	29
9.6	Example: structure first, arithmetic second	30
9.7	Example: semantic branch pruning	30
9.8	Skeleton replay and proof readability	30
10	Z3 services for proof writing	31
10.1	One solver, several narrowly specified services	31
10.2	A typed SMT query object	31
10.3	Counterexample service	31
10.3.1	Exact reflected fragments	32
10.3.2	Abstract models	32
10.3.3	Model validation	32
10.4	Unsat-core service	32
10.5	Quantifier-instantiation service	33
10.6	Template-solving service	33
10.7	Branch-feasibility service	34
10.8	Fixedpoint and invariant service	34
10.9	Solver process management	34
11	Theory-specific proof reconstruction	36
11.1	Why reconstruction should be theory-specific	36
11.2	Propositional reasoning and EUF	36
11.3	Linear integer arithmetic	36
11.4	Linear rational and real arithmetic	37
11.5	Bit vectors	37
11.6	Algebraic datatypes	37
11.7	Arrays, maps, and extensionality	38
11.8	Polynomial identities and nonlinear arithmetic	38
11.9	Quantifiers	38
11.10A	reconstruction registry	39
12	Cooperation with grind, TryThis, and premise selection	40

12.1	Three integration stages	40
12.1.1	Stage 1: sibling tactic portfolio	40
12.1.2	Stage 2: fact exchange	40
12.1.3	Stage 3: solver extension	40
12.2	How Leant can consume grind facts	41
12.3	How Z3 can consume grind facts	41
12.4	Premise selection loop	41
12.5	Suggestion ranking	42
12.6	A proposed solve? command	42
13	Quantifiers, higher-order structure, and theorem retrieval	43
13.1	The structural/semantic division is especially valuable here	43
13.2	Bounded rank-N planning	43
13.3	Monomorphization as a proof-producing transformation	43
13.4	Model-guided instance generation	44
13.5	Theorem retrieval beyond symbol overlap	44
13.6	Abduction and missing-lemma suggestions	45
14	Induction, recursion, invariants, and termination	46
14.1	Why induction is the next major frontier	46
14.2	Induction-skeleton synthesis	46
14.3	Invariant synthesis loop	46
14.4	Linear and lexicographic ranking functions	47
14.5	Predicate discovery from cores and models	47
14.6	Recursive data and fold lemmas	47
14.7	Statement strengthening	48
15	Counterexample-guided synthesis and proof repair	49
15.1	CEGIS for proof terms and witnesses	49
15.2	Semantic fingerprints and candidate equivalence	49
15.3	Proof repair after source changes	49
15.4	Diagnosing false or underspecified lemmas	50
15.5	Synthesis under behavioral constraints	50
15.6	Limits of finite counterexamples	50
16	Concrete protocol and API design	51
16.1	Design goals	51

16.2	Stable identities	51
16.3	Semantic term and type AST	51
16.4	Binder visibility and implicit information	52
16.5	Projection capabilities	52
16.6	Progress and truncation	53
16.7	Evidence payloads	53
16.8	Candidate rejection taxonomy	54
16.9	Schematic Lean reconstruction API	54
16.10	Schematic Haskell worker API	55
17	Scheduling, caching, and process isolation	56
17.1	A deterministic portfolio scheduler	56
17.2	Cancellation	56
17.3	Cache layers	57
17.4	Incremental sessions	57
17.5	Resource isolation	58
17.6	Deterministic replay bundles	58
18	User experience and proof presentation	59
18.1	Commands and their promises	59
18.2	Result wording	59
18.3	Readable proof selection	60
18.4	Interactive steering	60
18.5	Explanations from proof graphs	60
18.6	Proof drafts and helper lemmas	60
19	Implementation roadmap and acceptance gates	62
19.1	Principle of the roadmap	62
19.2	Milestone 0: shared fixtures and revision discipline	62
19.3	Milestone 1: native leant? using current candidate strings	62
19.4	Milestone 2: typed proof graphs and residual holes	63
19.5	Milestone 3: Z3 counterexamples and unsat cores	63
19.6	Milestone 4: theory-specific reconstruction	63
19.7	Milestone 5: cooperative solve? and premise feedback	63
19.8	Milestone 6: induction and invariant synthesis	64
19.9	Milestone 7: selective certificate and proof-trace replay	64

19.10 Recommended stopping points	64
20 Evaluation methodology	65
20.1 Metrics	65
20.2 Benchmark strata	65
20.3 Baselines	66
20.4 Metamorphic tests	66
20.5 Differential tests	67
20.6 Adversarial protocol tests	67
21 Risks, semantic traps, and non-goals	68
21.1 Semantic mismatch risks	68
21.1.1 Natural numbers versus integers	68
21.1.2 Partial operations	68
21.1.3 Prop versus Bool	68
21.1.4 Functions and extensionality	68
21.1.5 Datatypes and recursion	68
21.1.6 Type classes and coercions	68
21.1.7 Definitional equality and transparency	68
21.1.8 Quotients, proof irrelevance, and subsingletons	69
21.2 Engineering risks	69
21.3 Non-goals for the initial system	69
21.4 What success looks like	70
22 Prioritized recommendations	71
22.1 Recommendation 1: implement leant? before changing the search	71
22.2 Recommendation 2: make typed holes the next core abstraction	71
22.3 Recommendation 3: add Z3 first as a model/core service	71
22.4 Recommendation 4: reconstruct with native theory tactics	71
22.5 Recommendation 5: integrate through public facts before private solver state	71
22.6 Recommendation 6: preserve evidence/progress distinctions end to end	71
22.7 Recommendation 7: treat induction as a proof-plan problem	72
22.8 Recommendation 8: defer universal proof-object checking	72
22.9 Recommended first prototype in one page	72
23 Conclusion	73

A	Capability matrix	74
B	Illustrative end-to-end scenarios	75
B.1	Scenario A: higher-order structure plus arithmetic	75
B.2	Scenario B: false finite theorem	75
B.3	Scenario C: quantified algebraic theorem	75
B.4	Scenario D: loop invariant	75
B.5	Scenario E: proof repair	76
C	Source map	77
	Lean 4 source map	77
	Leant and Djex source map	77
	Z3 and comparison source map	78

Executive conclusions

1.1 Bottom line

Primary recommendation

Build a single proof-automation portfolio around a typed, source-mapped proof graph. Use Leant/Djex for structural proof construction, Z3 for semantic search and counterexample generation, and Lean’s existing tactics for normalization and proof reconstruction. Treat every external component as an untrusted producer of candidates or evidence. The only operation that closes a goal is construction of a Lean term accepted by the current elaborator and kernel, or execution of a small certificate checker whose soundness theorem is itself kernel-checked.

This division of labor matches the actual strengths of the three codebases:

- **Lean knows the exact dependent type theory, elaboration state, local instances, coercions, reducibility settings, universe constraints, and source context.** It should own projection, reconstruction, and final checking.
- **Leant/Djex knows how to construct inhabitants.** It is especially useful for introductions, higher-order use of hypotheses and providers, constructor and case structure, rank-N instantiation within explicit bounds, and short compositions that are awkward for first-order SMT encodings.
- **Z3 knows how to search semantic spaces.** It is especially useful for finding countermodels, eliminating impossible branches, selecting a small set of premises, proposing quantifier instances, solving arithmetic templates, minimizing choices, and synthesizing invariants or ranking parameters.
- **Lean’s native solvers know how to turn many of those facts into proofs.** `grind`, `omega`/linear-arithmetic extensions, `grobner`, simplification, congruence closure, and `bv_decide` already produce or check proof evidence in the target system.

The architecture should therefore optimize the exchange of *typed hints* rather than search for one universal solver.

1.2 The seven most valuable directions

Direction	What it adds	Soundness path	Priority
Native leant?/leant tactic	Makes structural synthesis available at any Lean goal; emits a pasteable proof suggestion.	Re-elaborate candidate in the saved tactic state; kernel checks final term.	strong

Direction	What it adds	Soundness path	Priority
Typed, hole-bearing skeletons	Lets Leant solve introductions, applications, cases, and constructors while leaving arithmetic or domain facts to other tactics.	Reconstruct with refine; every hole remains a Lean metavariable.	strong
Z3 counterexample and core oracle	Gives concrete failures, premise minimization, infeasible branch detection, and better diagnostics before expensive search.	Validate model decoding; treat cores as hints only.	strong
Theory-specific proof reconstruction	Converts Z3 discoveries into calls to Lean's EUF, arithmetic, bit-vector, array, and datatype proof infrastructure.	Lean tactic or small checker produces the proof.	strong
Cooperative solve? portfolio	Schedules Lean-native solvers, Leant, premise selection, and Z3 under deterministic budgets.	Each lane has its own evidence label; only checked proofs close.	strong
Induction and invariant synthesis	Leant chooses recursors and proof shape; Z3 solves coefficient, invariant, or ranking templates; Lean checks VCs.	Kernel checks base, step, postcondition, and decrease obligations.	medium
Generic Z3 proof-object translation	Potentially replays a wider SMT proof directly.	Requires a large, version-sensitive parser and independently proved checker.	defer

1.3 What should not be built first

Three tempting approaches have a poor cost-to-trust ratio as initial projects.

1. **Do not make unsat a primitive Lean axiom.** This would enlarge the trusted base to include the translator, solver binary, options, theory semantics, protocol, and process boundary. It would also make proof replay and regression diagnosis much worse.
2. **Do not start with a universal Lean-to-SMT translator.** Dependent types, type classes, coercions, partial operations, quotients, proof irrelevance, recursive definitions, and definitional equality make "translate everything" an endless source of semantic gaps. Begin with exact, named fragments and leave residual goals.
3. **Do not start by parsing every Z3 proof rule.** Z3's proof support is solver-configuration-dependent, broad, and evolving. For the first useful system, reconstruct results through Lean's own theory tactics and check small certificates such as LRAT where a mature checker already exists.

Positive results are cheap to trust; negative results are not

A proposed proof term can be completely untrusted: Lean either accepts it at the exact target or rejects it. By contrast, “no proof exists”, “this branch is impossible”, or “this model refutes the theorem” requires a precise projection contract. The system should preserve that asymmetry explicitly in every result type.

1.4 A concrete first result

The first implementation target should be a package tentatively named `Leant.Tactic` with the following behavior:

Listing 1.1: Proposed first user-facing command

```
import Leant.Tactic

example (A B C : Prop)
  (f : A -> B -> C) (g : A -> B) (a : A) : C := by
  leant?
-- Try this:
--   exact f a (g a)
```

The initial version may keep the existing Haskell worker and even the current rendered candidate format. The Lean tactic should snapshot the goal, invoke a revision-pinned worker, verify each candidate against the saved state, and use `TryThis` to show the first accepted script. That produces immediate value while establishing the transport, cancellation, stale-result, and test infrastructure needed by every later feature.

Scope, method, and repository snapshots

2.1 Exact revisions

The review was performed on 10 August 2026 against the revisions below. The Leant repository head contains report-only commits after its latest functional integration; where implementation details are discussed, source links use the latest functional commit [0160237e4bdf](#). Leant’s Djex submodule pin was read from the repository tree rather than inferred from documentation.

Repository	Pinned revision	Relevant state
leanprover/lean4	1d2dde6d64cb	Current development head at the time of review. Includes the modular grind engine, replayable grind?, library suggestions, symbolic tactic integration, LRAT-backed bv_decide, and the recent sharing of grind state with bv_decide.
Z3Prover/z3	0bd679a404be	Current development head. Exposes models, proofs, unsat cores, optimization, fixed-point/Horn solving, quantifiers, algebraic datatypes, arrays, sequences, bit-vectors, and configurable proof logging.
Leant	dab7a1518f51	Repository head. Latest functional source is represented by 0160237e4bdf , followed by documentation commits.
Djex	f50b5eac8672	Exact Leant submodule pin. Provides the checked Djinn and Exference adapters, common query/evidence vocabulary, bounded rank-N/provider support, and generated expression infrastructure.
ufmg-smite/lean-smt	a4ed29b0b9fd	Comparison point: active beta tactic that sends Lean goals to cvc5 and replays solver proofs in Lean, leaving unsupported proof steps as residual Lean goals.

2.2 Questions investigated

The investigation focused on five engineering questions.

1. Which current extension points in Lean can host external structural and semantic automation without patching the kernel or depending prematurely on unstable internals?
2. Which information does Leant/Djex already preserve, and which information is lost when a checked

candidate becomes rendered source text?

3. Which Z3 outputs can safely be used as proofs, proof plans, diagnostics, counterexamples, or search heuristics?
4. How can the systems cooperate on one proof state instead of merely racing as independent tactics?
5. What staged implementation produces useful results while keeping the trusted computing base small and every failure mode explicit?

The report is intentionally architecture-oriented. It reviews exact source seams and recent development direction, but it does not attempt a benchmark run or implement a prototype. Proposed APIs are schematic and are meant to make data ownership, verification, and versioning precise; exact names would naturally change during implementation.

2.3 Terminology

Candidate

An untrusted proposed Lean term, tactic script, proof graph, theorem instance, model, or solver-derived fact.

Reconstruction

Building a Lean proof from external hints using elaboration, theorem application, native tactics, or a verified checker.

Certificate

Data checked by a comparatively small Lean program whose soundness theorem implies the desired proposition.

Projection

A controlled translation from an elaborated Lean goal or subgoal to another calculus or semantic theory.

Residual goal

A Lean metavariable intentionally left by a partial proof skeleton for another tactic or the user.

Proof-producing portfolio

A scheduler in which heterogeneous engines may search aggressively, but every successful lane returns a kernel-checkable proof or checked certificate.

Lean 4 as the proof-producing host

3.1 The relevant architecture is already present

Lean’s current tactic infrastructure is not a thin wrapper over kernel primitives. It contains a substantial proof-producing search substrate. The most relevant components are shown in Figure 3.1.

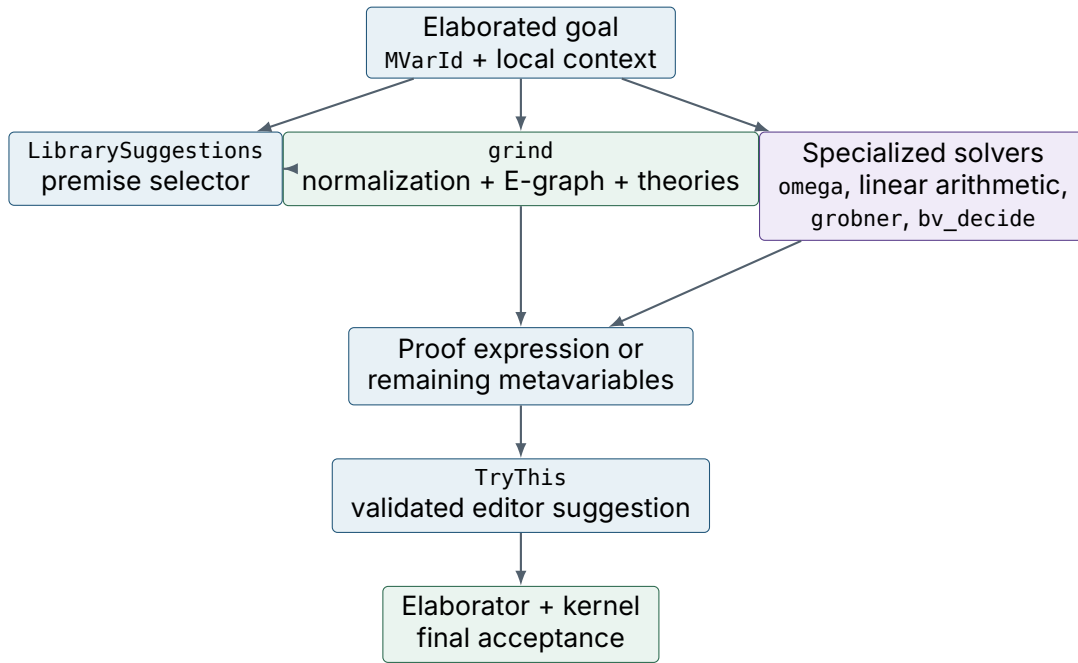


Figure 3.1: Lean’s existing substrate already separates premise selection, proof search, specialized decision procedures, validated suggestions, and kernel acceptance.

This matters because external tools do not need to own the tactic experience. They can enter at several narrower seams: selecting premises, proposing a proof skeleton, proposing derived facts, supplying a theory certificate, or generating a diagnostic model.

3.2 grind: a natural cooperation point

At the pinned revision, `grind` is a modular goal-level engine rather than a single opaque tactic. Its elaborator builds parameters from explicit theorem patterns, configuration, and library suggestions. The meta-level engine maintains a backtrackable state containing simplification data, congruence information, E-matching theorems and constraints, split sources, diagnostics, theory-solver state, and proof reconstruction information. Specialized front ends for linear integer arithmetic, linear arithmetic, order reasoning, and Groebner-style algebra use the same core.

Several details are particularly important for future integration:

- E-matching theorems carry origins, proof expressions, patterns, symbols, and bounded constraints

such as generation, size, depth, groundness, values, and a maximum number of instances.

- The engine receives an array of extension states. This shows that the implementation already thinks in terms of cooperating theory or theorem extensions, even though a stable third-party solver-extension API should not be assumed from current internal types.
- `grind?` does not merely print a heuristic recommendation. It constructs a self-contained script and validates it through the standard suggestion machinery before presenting it.
- The elaborator can call `LibrarySuggestions.select` with the caller name "grind", so premise retrieval is already a replaceable component rather than hard-coded theorem enumeration.

The most informative recent change is the integration of `bv_decide` with interactive `grind`. The bit-vector tactic can consume relevant equivalence classes and learned facts from the current `grind` state before bit-blasting. This is precisely the cooperation pattern a future Z3 or Leant bridge should emulate: share normalized facts and source identities, then let a specialized solver produce its own independently checked evidence.

Do not bind the first prototype to private `grind` internals

The `grind` implementation is moving quickly. Canonicalization, pattern handling, homomorphism support, symbolic execution, and theory integration have all changed recently. The first external integration should be a sibling tactic that exchanges ordinary Lean facts and residual goals. A true solver extension should follow only after a stable public interface exists or after the interface is designed together with Lean maintainers.

3.3 TryThis: the right user-interface seam

`Lean.Meta.Tactic.TryThis` already supports validated editor code actions. It can save and restore tactic state, check a proposed script, emit `exact` or `refine`, insert `expose_names` when generated references require stable names, and preserve remaining subgoals. This makes it an ideal endpoint for external automation:

- `leant?` can show a structural term or a `refine` skeleton.
- `z3?` can show a reconstructed proof plan, a minimal set of premises, or a counterexample diagnostic without changing the goal.
- `solve?` can compare candidates from several lanes and present the shortest validated script rather than the first raw solver response.

A crucial design rule follows: the external protocol should not be responsible for pretty-printing the final user script. It should return structured references and proof nodes. Lean should elaborate, normalize names, decide whether `exact` or `refine` is appropriate, and then ask `TryThis` to validate the final text.

3.4 LibrarySuggestions: premise selection as a first-class service

The core suggestion API maps a goal and configuration to scored theorem suggestions. The configuration records the caller, a maximum result count, filters, and an arbitrary hint. The API also provides combinators for combining and filtering selectors and utilities for collecting constants and relevant constants from a goal. No single premise-selection algorithm is privileged.

This is a strong insertion point for both Leant and Z3:

- Leant’s live provider inventory can be seeded by a selector instead of scanning a broad declaration prefix. The engine can still widen geometrically if the first ranked providers fail.
- A Z3 unsat core can feed back which translated premises were actually needed. Repeated results can train or heuristically adjust theorem scores without trusting the core as a proof.
- Model-based diagnostics can identify which missing relation or constructor fact blocks the current proof and request more premises sharing those symbols.
- The selector hint can carry a normalized goal signature or feature vector without changing the theorem-search API.

3.5 Reflected checking in `bv_decide`

The current bit-vector tactic is a model for trustworthy external acceleration. It bit-blasts a reflected bit-vector expression, converts the result to CNF, parses an LRAT certificate, and runs a Lean checker. The theorem `verifyCert_correct` connects a successful checker result to CNF unsatisfiability; another theorem connects the bit-blasted CNF back to the original bit-vector expression. The heavy SAT search is outside the trusted proof path.

This pattern should be copied whenever a compact certificate format is available:

$$\begin{aligned} \text{Lean goal} &\longrightarrow \text{reflected problem} \longrightarrow \text{external search} \longrightarrow \text{certificate} \\ &\longrightarrow \text{small Lean checker} \longrightarrow \text{kernel theorem.} \end{aligned}$$

It should not be generalized mechanically to every Z3 theory. LRAT works because the problem has a clean reduction to propositional SAT and a mature certificate format. For EUF, arrays, arithmetic, datatypes, and quantifiers, theory-specific reconstruction is likely smaller and more maintainable than one universal proof-object checker.

3.6 Implications for an external automation design

Lean should own the following state and decisions:

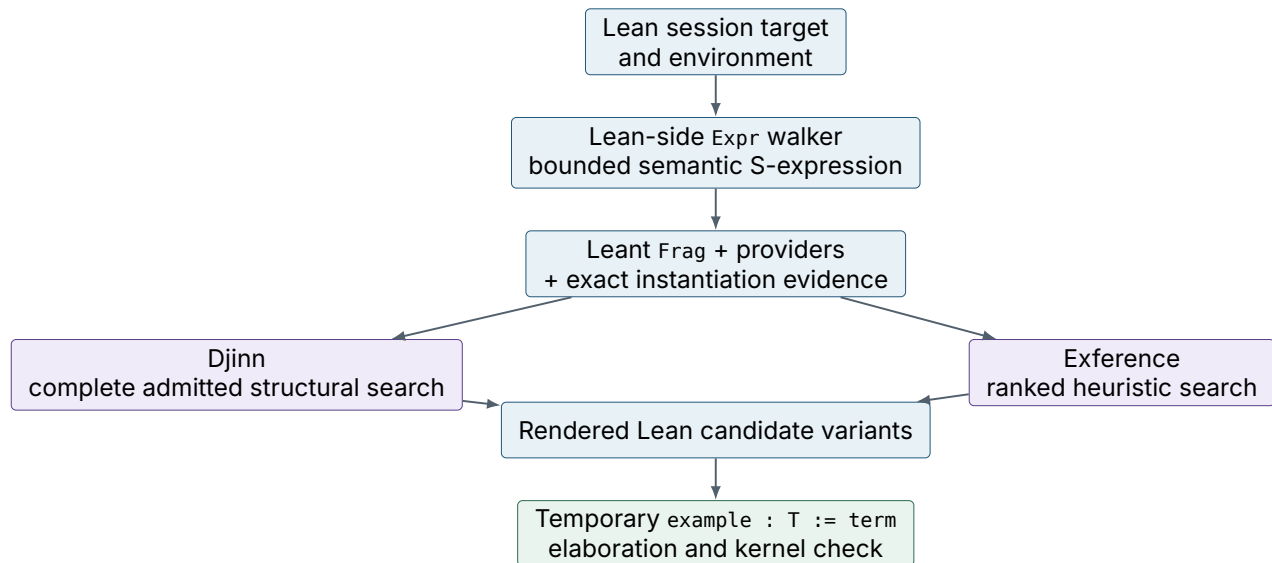
- exact target expression and local context;
- universe metavariables and constraints;
- binder information, local instances, coercions, and reducibility policy;
- declaration and theorem identities;
- source ranges and user-visible names;
- which residual goals are acceptable;
- reconstruction and final checking.

External workers should receive a bounded, versioned projection and return bounded, versioned evidence. They should never receive permission to close an `MVarId` merely by asserting success.

Leant and Djex as structural proof synthesizers

4.1 Current end-to-end path

Leant is already close to an external proof-synthesis service. Its present pipeline is approximately:



The final verifier is generated as source text with the essential shape:

Listing 4.1: Current Leant acceptance boundary

```

set_option autoImplicit true in
noncomputable example : (goal) := (candidate)

```

This is a valuable boundary: search, rendering, provider discovery, and even the protocol parser may be wrong without making Lean accept an ill-typed term. The same boundary should remain in the first native tactic, where it can be strengthened by elaborating directly in the saved tactic state rather than in a reconstructed REPL command.

4.2 The admitted structural fragment

Leant's Lean-side serializer elaborates the target and walks the resulting Expr. The Haskell-side Frag distinguishes:

- arrows, products, sums, top, bottom, and lowered forms of equivalence and negation;
- explicit and implicit quantification over sorts;

- instance-implicit binders that are erased for search but retained as rendering slots;
- exact, closed contextual class evidence attached to a particular provider assignment;
- opaque variables and atoms with a safety bit controlling negative claims;
- proper-type applications with variable or exact nominal heads;
- non-recursive and recursive inductive occurrences, including parameterized families and bounded constructor inventories;
- an explicit depth-truncation marker.

This is not a generic serialization of Lean syntax. It is a semantic projection into the part of intuitionistic type structure that Djex can exploit, with explicit markers for where structure was hidden or truncated. That distinction is exactly what future proof automation needs: projections should carry their loss, not merely a Boolean “supported” flag.

4.3 Evidence and progress are already separated

The public synthesis result distinguishes candidate production, refutation, and bounded failure:

Listing 4.2: Simplified current Leant outcome type

```
data SynthOutcome
  = SynthCandidates [[String]] [String]
  | SynthRefuted Bool
  | SynthNoTerm [String]
```

The Boolean on `SynthRefuted` records whether the projection was sufficiently complete and free of unsafe opaque atoms for the negative verdict to be advertised as sound. Exference never contributes negative evidence merely because its budget ended. Progress notes distinguish step, choice-point, candidate, queue, and depth limits.

This evidence/progress split should be preserved verbatim in a future shared protocol. It prevents a common automation bug: conflating “the search did not find a term” with “no term exists.” Z3 has the analogous three-way result `sat/unsat/unknown`; a unified API should not flatten either system into success/failure.

4.4 Current finite bounds are part of the contract

At the reviewed revision, Leant’s live exact-provider discovery deliberately uses small, testable bounds. Among the important limits are:

- at most six leading type binders opened on one provider;
- at most 32 active instance heads inspected in resolver order;
- at most 16 alpha-distinct assignment vectors retained per provider;
- at most 32 provider/vector associations passed into search;
- at most six arguments in one exact vector;
- a residual first-order kind-arrow bound of 64;
- a bounded vector of visible forall-domain tags;

- bounded candidate, display, and verification windows;
- bounded Exference queue and step policies.

The current functional history widened rank-N and provider instantiation from five to six binders while preserving seven binders as an explicit fail-closed boundary. It also added canonical higher-kinded Prod and Sum assignments at total arity two while retaining conservative failure for unsupported structural heads and legacy payloads. These are signs of healthy engineering: bounds are observable, tested, and associated with evidence strength.

A future native tactic should expose such bounds as structured diagnostics and cache keys. It should not silently vary them based on thread timing or solver load.

4.5 Djinn and Exference form a useful internal portfolio

Djinn and Exference have complementary semantics:

- DJINN performs a terminating, complete search for its admitted LJT-oriented structural fragment. When projection completeness is established, exhaustion can support a negative claim about that abstract problem.
- EXFERENCE performs ranked heuristic synthesis under explicit budgets. It is often better at useful compositions, but exhaustion carries no negative meaning.
- Combined mode uses a fair bounded merge rather than allowing one engine's repeated candidates to crowd the other out.
- Live providers are tried in bounded widening stages. Djinn can begin with the highest-ranked provider and expand geometrically; Exference retains its own rated full-inventory lane.

This is already the seed of the larger portfolio proposed in this report. Z3 should be added as an orthogonal semantic service, not as a replacement for either structural engine.

4.6 Where the current representation blocks deeper cooperation

The current public candidate is a group of rendered strings. That is sufficient for final re-elaboration, but it loses information needed by a cooperative tactic:

- stable identity of every local hypothesis and provider occurrence;
- the expected type at every subexpression;
- which type arguments were synthesized and why;
- constructor and case-branch source identities;
- residual constraints and holes;
- sharing between repeated subterms;
- provenance and ranking contributions;
- the exact projection loss relevant to this candidate;
- a place to attach Z3 models, cores, or branch-feasibility evidence.

The durable next representation is a typed candidate graph. Rendered Lean source should remain as an output view, not the semantic interchange format.

Most important Leant evolution

Change the narrow engine boundary from “fragment in, ranked strings out” to “typed goal snapshot in, ranked proof graphs with residual obligations out.” Keep the existing string renderer as one consumer and keep Lean re-elaboration as the mandatory acceptance boundary.

4.7 A native tactic need not move Djex into Lean

There is no requirement to rewrite Djex in Lean or bind the Haskell runtime into the Lean process. A long-lived worker process is arguably the better first architecture:

- it isolates crashes, memory growth, and cancellation;
- it can retain Djex sessions and provider caches across goals;
- it permits independent release and benchmarking;
- the protocol can be recorded for deterministic replay;
- Lean remains responsible for all trusted state and final checks.

The important change is the interface: a native Lean tactic should own the goal snapshot and user interaction, while Leant becomes a service that consumes a bounded projection and returns bounded structured candidates.

Z3 as a semantic search and diagnostic engine

5.1 The useful outputs have different logical strengths

Z3 exposes solver results, models, unsat cores, proof ASTs, optimization, and fixedpoint answers. These are not interchangeable.

Z3 artifact	Best use in Lean automation	What must happen before it can close a goal
sat model	Counterexample search, candidate refutation, concrete test generation, selector assignment.	Decode into the exact projected domain and re-evaluate all tracked assertions; for theorem-level counterexamples, prove or check realization in Lean.
Unsat core	Premise minimization, branch explanation, theorem-retrieval feedback, smaller replay problem.	Reconstruct a proof using the selected premises; the core itself proves nothing in Lean.
Quantifier instances / solver trace	Suggest explicit theorem applications and E-matching seeds.	Rebuild well-typed Lean terms for every instantiation and prove the resulting instances.
Proof AST / inference log	Diagnostic explanation; possible future rule-by-rule replay.	Parse a versioned rule language and check each step independently, or translate to another checked certificate.
Optimize model	Choose smallest premise set, cheapest candidate parameters, invariant coefficients, or branch selectors.	Verify the chosen values and all resulting obligations in Lean.
Fixedpoint answer / invariant	Propose loop invariants, Horn summaries, ranking relations, or unreachable states.	Materialize a Lean predicate and prove initialization, preservation, and post-condition/decrease obligations.
unknown + reason	Honest boundary information and fallback routing.	Never closes a goal. Any model returned after unknown is diagnostic only unless separately validated.

The C API documents the key caveats. Proof retrieval requires proof generation to have been enabled and a preceding unsat result. Unsat cores are not minimized by default; minimization options cost additional search. Models are supported after sat; after unknown, a returned model is not guaranteed to satisfy the assertions, especially quantified assertions. A sound integration should encode these conditions in constructors rather than rely on callers to remember them.

5.2 Z3's strongest early roles

The highest-value roles do not require parsing a full proof object.

5.2.1 Validated counterexample generation

For exact finite or first-order fragments, Z3 can refute a candidate or conjecture quickly. A useful result contains:

- the exact projection version and source map;
- assignments for source-level variables, not raw solver symbols;
- the values of tracked intermediate expressions;
- a validation record showing that every asserted formula evaluates to true and the negated goal evaluates to true in the decoded model;
- a warning when the model uses uninterpreted elements that cannot be rendered as concrete Lean data.

For bit vectors, finite enums, bounded naturals, tuples, and finite algebraic datatypes, model realization can be exact. For arbitrary functions, infinite types, quotients, or axiomatized structures, the result may be only an abstract first-order model. The UI must say which kind it is.

5.2.2 Assumption-based unsat cores

Every candidate premise can be guarded by a Boolean assumption literal:

Listing 5.1: Schematic tracked-premise encoding

```
(declare-const use_h0 Bool)
(declare-const use_h1 Bool)
(assert (=> use_h0 H0))
(assert (=> use_h1 H1))
(check-sat-assuming (use_h0 use_h1 goal_negated))
(get-unsat-core)
```

The resulting core can reduce the theorem set handed to `grind`, `Leant`, or a reconstruction tactic. It can also explain why a candidate branch is impossible. The core is not necessarily minimum and may vary with solver configuration, so the integration should distinguish *core*, *locally minimized core*, and *proven minimal core* if the latter is ever computed.

5.2.3 Semantic branch pruning

A partial proof or program skeleton may introduce Boolean selectors, constructor choices, arithmetic parameters, or finite branch conditions. Z3 can check whether the current accumulated constraints are satisfiable before `Leant` expands the branch. This is especially useful for Exference-like search, but it must be an effectful outer scheduler rather than an `unsafePerformIO` call hidden inside a pure search function.

5.2.4 Template solving and optimization

Linear invariant templates, ranking functions, finite lookup tables, branch guards, and arithmetic witnesses can be represented by unknown coefficients. Z3 can solve or optimize those coefficients. Lean

then receives concrete terms and proves the resulting verification conditions. This is a cleaner use of SMT than trying to translate an entire dependent proof.

5.3 Proof logging is useful but not yet the trust boundary

The pinned Z3 tree includes an example of inference logging, replay, and optional self-validation for a SAT+EUF-oriented configuration. The example explicitly notes that some quantifier instantiations can be syntactically checked while others are validated by another SMT call. This is valuable engineering support, but it is not an independent proof checker in Lean. The same implementation family is helping validate its own trace, and supported inference kinds depend on solver configuration.

A future proof-object project should therefore be scoped by theory and strategy. For example:

1. use Z3 only to select premises and instances;
2. reconstruct EUF equalities with Lean congruence lemmas;
3. dispatch arithmetic leaves to Lean arithmetic tactics;
4. dispatch propositional leaves to a checked SAT certificate;
5. leave unsupported rules as explicit residual goals;
6. record the original proof object for diagnostics but do not make it trusted.

This resembles the current `lean-smt` strategy with `cvc5`: translate a goal, replay what is supported, and expose proof holes as ordinary Lean goals. That precedent shows that partial replay is useful; it also argues against waiting for universal coverage.

5.4 Recent Z3 history reinforces the fail-closed design

Recent changes around quantifier elimination, higher-order matching, polymorphic TPTP translation, model-based projection, cross-platform determinism, and optimization soundness demonstrate both Z3's strength and the difficulty of treating a fast solver as a monolithic oracle. Several fixes explicitly downgrade unsupported dependent or polymorphic shapes to a "gave up" result rather than returning a potentially unsound theorem. Other changes address solver-path instability caused by hashes or C++ argument evaluation order.

The lesson is not that Z3 is unreliable; mature solvers improve precisely by finding and fixing such cases. The lesson is that Lean integration should be revision-pinned, strategy-pinned where reproducibility matters, explicit about unknown, and constructed so that a solver defect cannot create a kernel theorem without passing an independent check.

5.5 Candidate theory frontier

A pragmatic first theory frontier is:

Theory	Early Z3 role	Lean proof path	Phase
Propositional / EUF	Cores, instances, congruence plans, branch pruning.	grind, simplifier, explicit congruence/equality proof.	1

Theory	Early Z3 role	Lean proof path	Phase
Linear integer arithmetic	Models, infeasibility, coefficients, premise cores.	omega, lia, or grind arithmetic extension.	1
Bit vectors	Counterexamples, branch pruning, arithmetic simplification hints.	bv_decide and checked LRAT.	1
Finite datatypes	Concrete models, constructor choices, disjointness/injectivity facts.	Cases, constructor theorems, grind.	1
Arrays	Counterexamples and select/store lemma plans.	Rewriting plus extensionality; residual goals allowed.	2
Linear real/rational arithmetic	Models, cores, coefficients.	Lean linear arithmetic, depending on imports.	2
Quantified first-order logic	Instance suggestions and premise selection.	Explicit theorem applications + grind; no raw unsat trust.	2
Horn/fixedpoint	Candidate invariants and summaries.	VC generation and checked induction.	3
Nonlinear arithmetic	Counterexamples, bounded templates, occasional proof plans.	grobner, normalization, or residual goals; Z3 may return unknown.	3
Strings, sequences, regex, floating point	Diagnostics and domain-specific experiments.	Requires dedicated reflection/reconstruction libraries.	defer

Trust model and result-strength lattice

6.1 The trusted computing base

The minimal trusted base should remain:

- the Lean kernel and the declarations imported by the user’s environment;
- the elaborator and metaprograms used to construct the final proof term;
- any small certificate checker whose correctness theorem is used;
- the projection correctness lemmas needed by that checker.

Leant, Djex, Z3, the worker protocol, search heuristics, model printers, premise rankers, and proof renderers should be untrusted. They may cause failure, delay, or poor suggestions, but not an invalid theorem.

There is an additional policy question: Lean permits axioms and unsafe computation in the surrounding environment. A kernel-checked term may depend on axioms. The tactic should therefore distinguish *kernel accepted* from *axiom-free under a chosen environment policy*. Optional post-checks analogous to `##print axioms` can report dependencies, but they are separate from solver integration.

6.2 A result lattice, not a Boolean

A common protocol should use constructors with monotone logical strength. One possible ordering is:

1. **ProofAccepted**: a Lean term has been elaborated at the exact target and kernel-checked.
2. **CertificateAccepted**: a Lean checker accepted a certificate and its soundness theorem produced the target proof.
3. **ProofReconstructed**: external evidence was replayed into a Lean proof; after checking, this normally becomes **ProofAccepted**.
4. **SkeletonAccepted**: a Lean refine term was accepted but leaves typed residual goals.
5. **CounterexampleValidated**: a model was decoded and independently re-evaluated in an exact reflected fragment.
6. **AbstractModel**: a first-order model satisfies the projection but is not known to realize concrete Lean values.
7. **UnsatCoreHint**: a subset of translated assumptions used by the solver.
8. **NoCandidateWithinBounds**: a finite search frontier ended without a candidate.
9. **AbstractRefutation**: a complete abstract calculus refuted the projected sequent, together with a projection-adequacy label.

10. **Unknown:** timeout, resource limit, unsupported theory, incomplete quantifier search, interruption, or protocol failure.

Only the first two constructors close a goal directly. The third is an intermediate implementation state that should be immediately checked. The fourth intentionally changes the proof state. The remainder are diagnostics or search guidance.

6.3 Formal direction of projection guarantees

Let a Lean proof state be written informally as $\Gamma \vdash T$, and let P project it to an external problem.

For *positive proof search*, no global theorem about P is required if every returned candidate t is elaborated against the original target:

$$\text{elab}_{\Gamma}(t) : T \implies \Gamma \vdash t : T.$$

Projection bugs merely lose candidates or waste time.

For an external *unsatisfiability* result to prove the Lean goal without replay, one needs a soundness/adequacy bridge of the form

$$\text{Lean counterexample to } (\Gamma \vdash T) \implies \text{model of } P(\Gamma \wedge \neg T).$$

Then external unsatisfiability rules out every Lean counterexample admitted by the stated scope. Establishing such a theorem for general dependent type theory is much harder than checking a proposed term. This is why proof reconstruction should be the default.

For an external *sat model* to be a concrete Lean counterexample, the required direction is different: the model must decode to actual Lean values and evaluation must commute with the projection. This can be proved for reflected finite domains, bit vectors, and selected first-order datatypes; it should not be assumed for opaque uninterpreted sorts.

6.4 Projection reports

Every projected query should carry a structured report, for example:

Listing 6.1: Schematic projection report

```
structure ProjectionReport where
  exactConnectives      : Array SourceId
  opaqueAtoms           : Array SourceId
  erasedTypeclassBinders : Array SourceId
  monomorphizedDecls    : Array Name
  approximations        : Array Approximation
  depthTruncations      : Array SourceId
  unsupportedTerms       : Array UnsupportedTerm
  modelRealization       : ModelRealizationLevel
  negativeAdequacy       : NegativeAdequacyLevel
```

A single `Bool` named `complete` is not enough once multiple solvers and subgoals are involved. One approximation may be harmless for positive term re-elaboration but fatal for a countermodel or abstract refutation.

6.5 Protocol and parser trust

Even when external outputs are untrusted, protocol parsers must be hardened because they run inside or adjacent to the theorem prover process. The protocol should have:

- an explicit schema version and capability negotiation;
- exact Lean, Leant, Djex, and solver revision identifiers;
- bounded node counts, nesting depth, string length, integer width, and vector length;
- no executable Lean source embedded in semantic metadata;
- stable numeric IDs rather than relying on pretty-printed names;
- a cryptographic or collision-resistant hash of the goal snapshot;
- rejection of stale results when the goal or environment revision changes;
- explicit unknown, cancellation, timeout, and partial-result messages;
- deterministic replay logs suitable for regression tests.

Leant's current bounded S-expression parser and fail-closed exact-provider metadata are a good precedent. The proposed typed protocol should preserve that discipline.

Unified proof-automation architecture

7.1 Overview

The proposed system has five layers, shown in Figure 7.1.

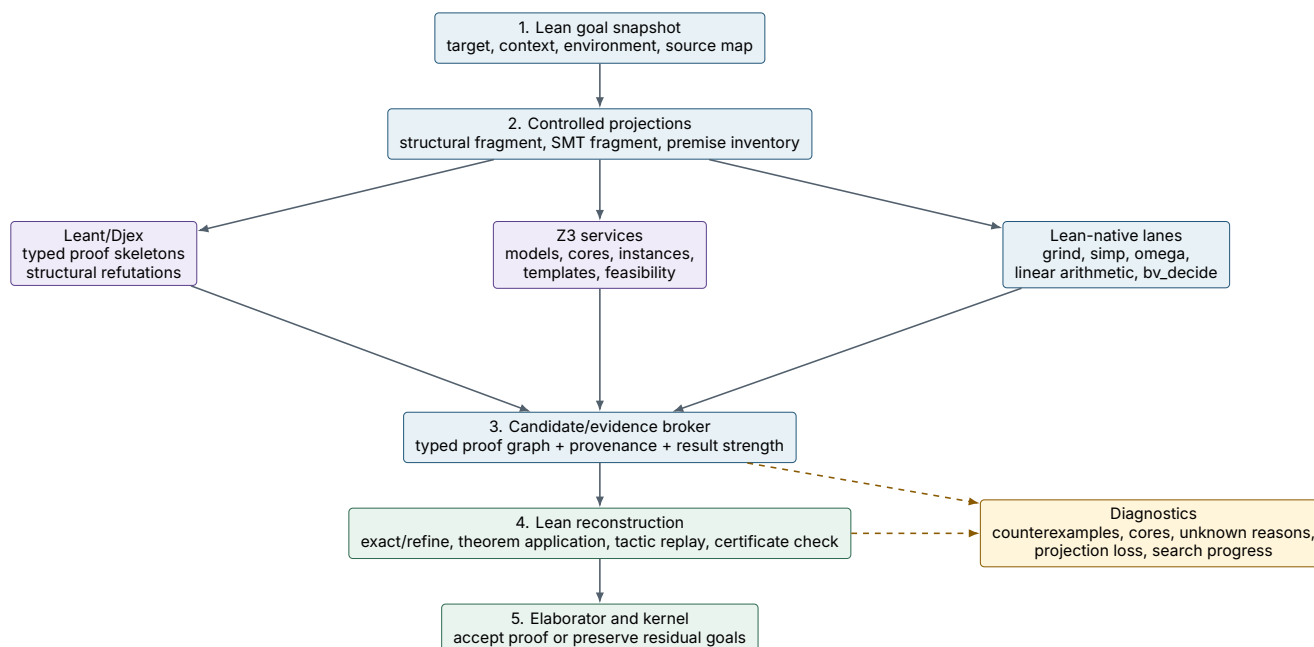


Figure 7.1: Recommended architecture. External engines never mutate the proof state directly; they return typed artifacts to a Lean-owned broker and reconstruction layer.

The key object is not an SMT formula or a rendered term. It is a *goal snapshot* with several projections and a *typed proof graph* whose holes can be delegated to different solvers.

7.2 Goal snapshots

A snapshot must contain enough information to reject stale or ill-scoped results while remaining serializable. One schematic Lean-side representation is:

Listing 7.1: Schematic Lean-owned goal snapshot

```

structure GoalSnapshot where
  snapshotId      : UInt64
  target          : Expr
  localContext    : LocalContext
  localInstances  : LocalInstances
  levelParams     : Array Name
  metavariableCtx : MetavarContext
  transparency    : TransparencyMode
  
```

```
environmentHash : ByteArray
sourceMap       : SourceMap
options         : Options
dependencySlice : Array Name
```

Not all fields should cross the process boundary. The worker receives a projection whose nodes refer to stable `SourceIds`. The full `Expr`, local context, and metavariable context remain in Lean and are used during reconstruction.

A dependency slice is preferable to hashing the entire environment. It can include the declarations referenced by the target, local hypothesis types, selected premises, and provider signatures. This gives meaningful cache invalidation without requiring a complete module hash. For strict reproducibility, the replay log should also record the Lean executable revision and imported module fingerprints.

7.3 A common candidate and evidence broker

Each lane should return an artifact with common metadata:

Listing 7.2: Schematic common external result

```
structure ExternalCandidate where
  snapshotId : UInt64
  candidateId : UInt64
  payload     : CandidatePayload
  provenance  : Provenance
  projection  : ProjectionReport
  evidence    : ExternalEvidence
  ranking     : RankingData
  progress    : ProgressReport
```

`CandidatePayload` may contain a proof graph, a set of derived facts, a theorem instantiation plan, a model, a premise core, or an invariant template. The broker does not infer logical strength from the producer name; the producer must supply an evidence constructor that the Lean side validates structurally.

The broker is also where duplicates are removed. Deduplication should use normalized semantic nodes and source identities before display limits are applied. Lean already deduplicates candidate spellings before bounded scheduling; the shared broker should generalize this to proof graphs and derived facts.

7.4 Typed proof graphs

A graph rather than a tree permits sharing of repeated subproofs and common residual obligations. A simplified node vocabulary is:

Listing 7.3: Schematic proof-graph vocabulary

```
inductive ProofNode
| exact      (term : TermRef)
| intro      (binder : BinderRef) (body : NodeId)
| apply      (function : TermRef) (arguments : Array NodeId)
| constructor (ctor : Name) (fields : Array NodeId)
| cases      (scrutinee : FVarRef) (branches : Array CaseBranch)
| have       (type : TypeRef) (proof body : NodeId)
| rewrite     (rule : TheoremRef) (direction : Bool) (body : NodeId)
| induction   (major : FVarRef) (motive : TypeRef)
```

```

    (branches : Array CaseBranch)
  | tactic      (capability : Capability) (inputFacts : Array FactRef)
  | hole       (holeId : HoleId) (expected : TypeRef)
                (context : ContextSlice) (capabilities : CapabilitySet)

```

```

structure ProofGraph where
  nodes      : Array ProofNode
  root       : NodeId
  sharing    : Array SharingConstraint
  sourceMap  : SourceMap

```

The graph uses stable references, not embedded pretty-printed Lean expressions. A `TermRef` may designate a local hypothesis, global declaration, literal, constructor, or a small source-independent term AST. During reconstruction, Lean resolves every reference in the saved context and checks every expected type.

7.5 Residual obligations are a feature

Traditional external tactics are often all-or-nothing: either the solver closes the goal or it reports failure. Typed holes make partial automation compositional. A Leant skeleton may solve the higher-order shape while delegating an arithmetic leaf to `omega`; `Z3` may identify two required instances while leaving an array extensionality lemma to `grind`; an induction planner may construct the recursor while leaving the invariant step as a user-visible goal.

A reconstruction result can therefore be:

Listing 7.4: Schematic reconstruction result

```

inductive ReconstructionResult
  | closed   (proof : Expr) (audit : AuditTrail)
  | refined  (proof : Expr) (goals : Array MVarId) (audit : AuditTrail)
  | rejected (reason : ReconstructionError) (audit : AuditTrail)

```

This representation aligns naturally with `TryThis`: a closed result becomes an exact suggestion; a refined result becomes a refine suggestion whose remaining goals are checked in the restored tactic state.

7.6 Capability-directed hole routing

Each hole should have a cheap syntactic and semantic capability analysis:

- propositional/EUF;
- linear integer arithmetic;
- linear ordered-ring arithmetic;
- polynomial identity;
- bit-vector decision problem;
- finite datatype case split;
- array select/store;
- structural term synthesis;

- induction or recursion required;
- theorem retrieval required;
- unsupported/dependent.

The scheduler can then route a hole to a small set of plausible lanes instead of running every engine. Capability analysis must be conservative: “unknown” means the hole may still be attempted by generic Lean tactics, not that it is unprovable.

7.7 Proof plans and diagnostics share source identities

The same `SourceId` should identify a local hypothesis in:

- the structural Leant fragment;
- the SMT declaration and tracked assertion;
- a library-suggestion item;
- a grind theorem or fact;
- an unsat core;
- a proof-graph application node;
- the final user-visible source range.

This avoids fragile name matching. It also lets the UI say, for example, “Z3 used hypotheses h1, h4, and h7; Leant’s candidate uses h1 and h4; grind closed the remaining arithmetic goal using h7.”

A native Leant tactic

8.1 Why this is the highest-leverage first step

Leant's current REPL proves that the core search and verification path works, but a separate session has unavoidable friction:

- the user must restate or replay the exact context;
- source locations and editor actions are indirect;
- the final candidate is checked in a temporary command rather than the exact saved tactic state;
- residual goals cannot be returned naturally;
- cooperation with grind, local tactics, and premise selectors is awkward;
- cancellation and stale-result handling are not integrated with the editor.

A native tactic removes all of these without changing the synthesis algorithms.

8.2 Proposed commands

The initial package can expose four commands:

leant?

Search and show one or more validated pasteable suggestions. It never changes the proof state unless the editor action is accepted.

leant

Apply the highest-ranked candidate that verifies in the current state. If the candidate is a skeleton, leave residual goals.

leant (engine := djinn|exference|both)

Select the structural engine and expose bounded search options.

leant_trace

Produce a diagnostic report containing projection loss, provider inventory, candidate rejection reasons, and search progress without modifying the goal.

A later `solve?` command can include Leant as one lane, but Leant should first be independently usable and testable.

8.3 First implementation: preserve the existing textual boundary

The minimum viable tactic can deliberately avoid a new proof-graph protocol:

1. Snapshot the active `MVarId`, target, local context, options, and source location.

2. Use Lean metaprogramming to emit the same bounded semantic fragment that the REPL serializer currently emits.
3. Send the fragment, provider inventory, and exact revision metadata to a long-lived `leant-server` process.
4. Receive bounded candidate strings and structured progress data.
5. Restore the saved tactic state and elaborate each candidate against the exact target with no sorry and normal kernel checking enabled.
6. Convert accepted candidates into `TryThis` suggestions.
7. Reject the entire response if the snapshot ID or environment fingerprint is stale.

This approach reuses the existing acceptance argument. It also supplies an immediate regression oracle for the later structured protocol: both paths should accept the same candidate set on the existing transcript suite.

8.4 Second implementation: return structured candidates

The next protocol revision should return a typed proof graph. The Lean side then constructs an `Expr` directly or builds a syntax term only at the final suggestion stage. Direct expression construction has several advantages:

- no accidental name capture or pretty-printer dependence;
- exact local declaration identities;
- explicit type argument and universe handling;
- natural creation of metavariable holes;
- better source-level explanation of rejected nodes;
- graph sharing and common subproofs;
- stable comparison of semantic candidates across engines.

The renderer remains important: users need readable source and replayable scripts. But rendering becomes Lean's responsibility after the graph has already reconstructed successfully.

8.5 Worker protocol

A line-delimited JSON or length-prefixed binary protocol is suitable. JSON is easier to record and inspect initially; a compact binary encoding can follow if profiling shows it matters. A request might have the following conceptual form:

Listing 8.1: Illustrative Leant worker request

```
{
  "schema": 1,
  "snapshot": "9f38...",
  "leanRevision": "1d2dde6d64cb...",
  "leantRevision": "0160237e4bdf...",
  "djexRevision": "f50b5eac8672..."
```

```

"goal": { "sort": "prop", "fragment": { ... } },
"providers": [ ... ],
"premises": [ ... ],
"engine": "both",
"limits": {
  "candidateWindow": 60,
  "verificationWindow": 24,
  "exferenceSteps": 128
}
}

```

Every response should repeat the snapshot ID and include a monotonically increasing response sequence. Streaming candidates is useful: Lean can verify the first few while the worker continues searching, then cancel when a satisfactory proof is found. A candidate message should not imply final search exhaustion; a separate terminal progress message should report completion or truncation.

8.6 Provider and premise selection

The native tactic can improve Leant's current provider path in two stages.

8.6.1 Ranked seed inventory

Ask `LibrarySuggestions` for theorem and definition candidates using a caller name such as `"leant"`. Preserve exact source names and provider-local instance metadata. Run Leant's established geometric widening over the ranked prefix.

8.6.2 Demand-driven expansion

If a candidate skeleton contains a hole headed by a nominal type or relation, query for additional providers whose result type or theorem conclusion shares that head. This avoids serializing a large environment before any structural search occurs.

The provider identity rules in current Leant should remain strict. Exact instance assignments are attached to the provider that exposed them; another provider with an alpha-identical type may not borrow the evidence.

8.7 Verification in the exact tactic state

The REPL's temporary example is strong, but a native tactic can be stronger and more precise:

- elaborate under the saved local context and metavariable context;
- require the elaborated type to be definitionally equal to the target;
- preserve the user's transparency and option settings where semantically appropriate;
- reject unresolved synthetic opaque metavariables except intentional skeleton holes;
- restore the original state after a rejected candidate;
- invoke the ordinary kernel type checker; never enable `debug.skipKernelTC`;
- record imported axioms and unsafe declarations separately for audit output.

Candidate verification should run under a bounded heartbeat and allocation budget so an adversarial or accidentally enormous term cannot freeze the editor.

8.8 Compatibility strategy

Leant's current live transcripts are verified against Lean 4.31, while the reviewed Lean head is later development code. The native package should therefore separate:

- a thin version-specific Lean projection/reconstruction module;
- a versioned, mostly stable semantic protocol;
- the independent Haskell search worker;
- golden protocol fixtures that can be replayed without starting Haskell;
- an integration matrix over supported Lean releases and development heads.

This localizes API drift in `Expr`, `elaborator`, `module`, and `tactic` internals.

Typed, hole-bearing proof synthesis

9.1 From complete terms to proof skeletons

A complete Leant candidate may fail because one small leaf lies outside the structural fragment. Typical examples include:

- a numeric inequality needed after applying the correct hypothesis;
- a field equality after choosing the correct constructor and cases;
- a definitional simplification involving a recursive function;
- a typeclass or coercion fact that Lean can synthesize but Djex intentionally erases;
- an array or finite-map lemma;
- an induction branch whose propositional shape is easy but whose invariant is semantic.

An all-or-nothing synthesizer discards the useful structural work. A skeleton should instead preserve the decisions it can justify and insert typed holes at unsupported leaves.

9.2 Hole creation rules

A hole should be created only at a boundary that Lean can independently check. Good boundaries include:

- the premise of an application whose function and result type are known;
- a constructor field;
- a case branch result;
- the proof of a local have;
- an equality or proposition after normalization;
- an induction motive or branch obligation;
- a typeclass obligation left to ordinary synthesis.

A hole should record its expected type, local context slice, source path, and the structural choices that depend on it. It should not merely be an untyped placeholder in a string.

9.3 Skeleton search objectives

Once holes are admitted, candidate ranking must account for them. A useful lexicographic objective is:

(unreconstructable holes, hole count, aggregate hole complexity,
external solver cost, proof graph size, provider penalty).

A complete but enormous term may rank below a short skeleton with one trivial arithmetic hole. Conversely, a skeleton with an opaque dependent hole should rank below a complete structural proof.

Hole complexity can combine:

- target AST size;
- quantifier alternation and universe dependence;
- theory classification;
- number of relevant hypotheses;
- presence of recursive definitions;
- expected solver availability;
- historical reconstruction success for similar goals.

These features affect ranking only; they never affect soundness.

9.4 Cooperative completion

The broker can attempt holes in dependency order:

1. run cheap definitional reduction and simplification;
2. invoke typeclass synthesis for instance holes;
3. run a matching native decision procedure for exact fragments;
4. ask Z3 for a counterexample/core/instance plan if semantic guidance is useful;
5. ask grind with the reduced premise set;
6. ask Leant recursively for a structural subproof if the hole exposes new structure;
7. stop at a user-visible residual goal if no checked lane succeeds.

The order should be configurable but deterministic. Each successful hole proof is a normal Lean expression and can be cached independently.

9.5 Shared subgoals and DAG search

Case analysis and theorem application often generate repeated obligations. A proof graph can hash normalized hole targets together with the relevant local context slice. Equivalent holes share one solver task and one proof node. This is particularly useful for:

- symmetric constructor branches;
- repeated arithmetic side conditions;
- quantified theorem applications at the same type arguments;
- induction branches with identical invariant facts;
- proof repair after a declaration rename.

Context equivalence must be conservative. Alpha-renamed local declarations can be canonicalized, but extra hypotheses may change typeclass search, reducibility, or simp behavior. The cache key should include the selected dependency slice rather than only the target text.

9.6 Example: structure first, arithmetic second

Consider a schematic theorem whose useful proof is “apply the only function that can produce the result, build its product argument, then prove a numeric bound.” Leant can produce:

Listing 9.1: Illustrative reconstructed skeleton

```
by
  intro f h x
  refine f x ?_
  -- goal: index x < bound
```

The hole classifier recognizes linear natural-number arithmetic. The broker first tries omega; if that fails, it may ask Z3 for a model or premise core, then retry omega or grind with the selected facts. The final proof contains no reference to Z3 or Leant unless the user chooses to keep the high-level tactic call.

9.7 Example: semantic branch pruning

Suppose Exference considers two constructors whose fields induce arithmetic constraints. The search graph can emit a partial branch formula for each constructor. Z3 proves one formula unsatisfiable under the current assumptions. This does not close the Lean goal, but it can prune that synthesis branch because the final candidate from that branch would necessarily fail the separately checked side conditions.

To remain safe under projection loss, branch pruning should use one of two policies:

1. **Exact policy:** prune only when the branch formula is an exact reflection of the side conditions and the unsat result is reconstructed or checked.
2. **Heuristic policy:** lower the branch priority but do not delete it. If the projection was incomplete or the solver result cannot be checked, search may still revisit the branch.

The second policy allows aggressive experimentation without sacrificing completeness claims of the structural engine.

9.8 Skeleton replay and proof readability

A proof graph should support at least three renderings:

- a compact term, suitable for exact;
- a tactic-style intro/refine/cases script with named residual goals;
- an audit view showing provenance, solver calls, core premises, and projection loss.

The shortest term is not always the best maintained proof. The ranking layer should permit a “readable” mode that prefers named intermediate facts, stable theorem applications, and native tactics over enormous proof terms.

Z3 services for proof writing

10.1 One solver, several narrowly specified services

The integration should not expose a single operation named `solveWithZ3`. It should expose services with distinct contracts:

1. `findCounterexample`;
2. `findUnsatCore`;
3. `proposeInstances`;
4. `solveTemplate`;
5. `checkBranchFeasible`;
6. `proposeInvariant`;
7. `obtainProofTrace` for diagnostics or experimental replay.

Each service uses a theory-specific projection, result type, validator, and resource policy.

10.2 A typed SMT query object

A query should be built from a typed internal SMT AST, not concatenated text. SMT-LIB may remain the process transport, but the builder should know sorts and source identities:

Listing 10.1: Schematic SMT query object

```
structure SmtQuery where
  logic      : SmtLogic
  declarations : Array SmtDecl
  assertions  : Array TrackedAssertion
  assumptions : Array AssumptionLiteral
  sourceMap   : SmtSourceMap
  timeoutMs   : Nat
  rlimit      : Option Nat
  randomSeed  : UInt32
  requestModel : Bool
  requestCore  : Bool
  requestProof : Bool
  strategy     : SolverStrategy
```

The query builder should reject ill-sorted terms before starting the process. The serialized script should set every non-default solver option explicitly when reproducibility matters.

10.3 Counterexample service

10.3.1 Exact reflected fragments

The first counterexample checker should support types with straightforward finite or computable semantics:

- `Bool`, finite enumerations, and finite products/sums;
- fixed-width `BitVec`;
- bounded `Fin n`;
- integers and naturals where all operations have an exact encoding and returned numerals fit the decoder;
- non-recursive algebraic datatypes and bounded recursive values;
- arrays over finite domains when decoded extensionally or as finite stores.

For each supported operator, the reflection layer needs an evaluation theorem or a trusted computation path checked by Lean. The service can then return a concrete assignment and a theorem or decidable check showing that the hypotheses hold and the goal does not.

10.3.2 Abstract models

For uninterpreted sorts and functions, Z3 may return a finite abstract universe and function tables. This is still useful: it often exposes a missing hypothesis or false algebraic intuition. The result should be labeled `AbstractModel`, rendered with stable element names, and should not be described as a concrete Lean counterexample unless a realization bridge exists.

10.3.3 Model validation

Validation should happen independently of the pretty printer:

1. parse the model into a typed model AST;
2. verify that every declaration has the expected sort and arity;
3. evaluate all ground tracked assertions with a separate evaluator;
4. for quantifier-free finite domains, enumerate quantified binders if bounded;
5. reject the model if any required value is absent or outside the supported decoder;
6. only then map values back to source-level terms.

This catches protocol errors and many solver-configuration mistakes. It does not replace a proof of the reflection semantics, but it makes the result robust.

10.4 Unsat-core service

The core service has three modes:

Fast core

Return the solver's assumption core with no minimization.

Deletion-minimized core

Iteratively remove assumptions and recheck. This is deterministic under a fixed order and often produces much smaller replay problems.

Weighted core

Use selectors and optimization or bounded search to prefer stable/local/user-specified premises. This need not find a globally minimum core to be useful.

A core can guide:

- which hypotheses are passed to grind;
- which library theorems are retained in a Leant provider stage;
- which facts are shown in a human-readable explanation;
- which proof dependencies are recorded in a repaired proof;
- which assumptions are likely redundant in the theorem statement.

The reconstruction layer should always be allowed to use fewer or more premises than the core. The core is a search hint, not a dependency certificate.

10.5 Quantifier-instantiation service

Z3's E-matching and model-based quantifier mechanisms can discover useful ground instances. The integration should export them as a plan:

Listing 10.2: Schematic theorem instantiation plan

```
structure InstantiationPlan where
  theorem      : TheoremRef
  typeArgs     : Array TermRef
  explicitArgs : Array TermRef
  triggerTerms : Array SmtTermId
  origin       : InstantiationOrigin
  dependencies : Array FactRef
```

Lean reconstructs the actual theorem application using its own elaborator. It checks universe levels, implicit arguments, coercions, typeclass synthesis, and definitional equality. If the instance is valid, the resulting proposition can be added to grind or the local context. If reconstruction fails, the plan is discarded and the failure reason is recorded.

This service is particularly complementary to Leant. Leant can decide where a higher-rank hypothesis must be applied; Z3 can suggest the ground arguments that make its first-order consequence useful.

10.6 Template-solving service

A template consists of a Lean-owned syntax with unknown coefficients or finite choices. Examples include:

- $a_0 + a_1x_1 + \dots + a_nx_n \geq 0$ as a linear invariant;
- a lexicographic ranking tuple;

- a polynomial equality witness;
- a finite decision tree over predicates;
- a branch selector for a finite set of lemmas;
- a bounded lookup table or finite function;
- numeric parameters in a constructive witness.

The service returns concrete coefficients and an objective value. It does not return a proof. Lean substitutes the values into the template and generates the exact proof obligations. Optimize can minimize coefficient magnitude, number of nonzero terms, number of selected premises, or a weighted syntactic cost.

10.7 Branch-feasibility service

Partial proof graphs can accumulate semantic constraints. The branch service should accept only a monotone conjunction of tracked facts and return:

- Feasible with an optional model;
- Infeasible with a core and a reconstruction status;
- Unknown with a reason and consumed budget.

An infeasible result may be used to prune only if the reconstruction status is strong enough for the search's completeness policy. Otherwise it changes ranking but not the set of explored branches.

10.8 Fixedpoint and invariant service

Z3's fixedpoint engine can solve Horn clauses and often return inductive invariants or answers useful for reachability. The integration should not attempt to import the entire answer formula blindly. Instead:

1. Lean generates Horn clauses from a selected recursive/loop specification.
2. The projection records each relation, state component, and transition source.
3. Z3 proposes a relation interpretation or invariant formula in a restricted template language.
4. Lean parses that formula into a typed predicate.
5. Lean proves initialization, preservation, exit/postcondition, and termination obligations using native tactics and, recursively, the semantic services.

If the answer uses unsupported constructs, it remains a diagnostic. Template-based invariant extraction is easier to trust and render than arbitrary solver formulas.

10.9 Solver process management

For the first version, a separate Z3 process speaking SMT-LIB is preferable to a C FFI:

- process termination is a reliable cancellation fallback;
- memory and CPU limits can be enforced by the operating system;

- exact scripts can be recorded and replayed;
- solver upgrades do not require rebuilding a Lean native extension;
- malformed or crashing solver behavior is isolated.

A pool of long-lived workers can still use incremental push/pop. Each worker must have a strict state machine so one cancelled query cannot leak assertions into the next. Periodic worker replacement avoids unbounded memory retention.

Theory-specific proof reconstruction

11.1 Why reconstruction should be theory-specific

A generic SMT proof is an interleaving of propositional reasoning, congruence, rewriting, preprocessing, quantifier instantiation, and theory lemmas. Replaying every internal rule couples Lean to solver implementation details. A smaller design classifies the final obligations and reconstructs each theory with the proof machinery Lean already has.

The external solver is then a *proof-plan generator*. It may identify a useful set of facts, a contradiction core, an equality path, or concrete coefficients. Lean constructs the actual proof.

11.2 Propositional reasoning and EUF

For quantifier-free propositional logic with equality and uninterpreted functions, a reconstruction pipeline can be:

1. Normalize source propositions into Lean’s grind representation while retaining theorem and source IDs.
2. Ask Z3 for a core, relevant equalities, and optional ground instances.
3. Add reconstructed instances as ordinary Lean facts.
4. Invoke grind on the reduced fact set.
5. If grind fails, reconstruct an explicit congruence path or expose the equality subgoal.

An equality explanation can be represented as a small DAG:

Listing 11.1: Schematic equality explanation

```
inductive EqReason
| hypothesis (fact : FactRef)
| symmetry (reason : ReasonId)
| transitivity (left right : ReasonId)
| congruence (function : TermRef) (arguments : Array ReasonId)
| rewrite (theorem : TheoremRef) (arguments : Array TermRef)
```

Lean checks every edge by theorem application. This format is independent of Z3’s internal proof node names and can also be produced by another EUF engine.

11.3 Linear integer arithmetic

Z3 is excellent at finding a contradictory subset, a model, or useful bounds. Lean’s native arithmetic tactics are the preferred proof path:

- omega for Presburger-style natural and integer arithmetic;

- `lia` or the linear arithmetic extension of `grind`, depending on the exact target and imports;
- simplification and normalization before arithmetic dispatch.

The reconstruction plan should map every SMT arithmetic atom back to its Lean expression. A Z3 core selects the candidate inequalities; Lean reruns the native tactic on those facts. If the native tactic succeeds, the proof is independent of the solver result. If it fails because the projection used an unsupported operation or a different normal form, the result remains a diagnostic.

For an arithmetic witness, Z3 may return a numeral or coefficient vector. Lean constructs the witness term and proves the required inequalities. This is often more useful than importing a contradiction proof.

11.4 Linear rational and real arithmetic

The same pattern applies to ordered rings and fields, with two additional cautions:

- Lean and SMT must agree on division, inverses, coercions, and side conditions such as nonzero denominators.
- The available reconstruction tactic depends on imported libraries. A core Lean package should not assume all of `mathlib`, but an optional `mathlib` adapter can use `linarith`, `nlinarith`, `normalization`, and `ring` tactics.

The projection should make coercions explicit and reject mixed arithmetic it cannot normalize soundly.

11.5 Bit vectors

Bit vectors have the cleanest proof-producing path:

1. Reify the Lean bit-vector proposition.
2. Use `grind` facts and Z3 only to simplify, find counterexamples, or select relevant hypotheses.
3. Run `bv_decide` on the resulting exact proposition.
4. Check the LRAT certificate in Lean.

Z3 need not be the SAT solver used by `bv_decide`; its role can be purely semantic and diagnostic. If a future Z3 path emits a compatible CNF certificate, it can feed the same checker, but that is an optimization rather than a design requirement.

The recent `bv_decide`/`grind` integration is a template for sharing facts: `grind` normalizes equalities and theory information, then `bv_decide` encodes the relevant equivalence classes into its reflected problem.

11.6 Algebraic datatypes

For finite and algebraic datatypes, Z3 can suggest constructor equalities, disjointness, injectivity, and case splits. Lean reconstruction should use the exact inductive declaration:

- constructor applications are rebuilt with universe and parameter arguments;
- disequality of distinct constructors is proved by no-confusion or generated disjointness theorems;

- equality of same-constructor applications is decomposed with injectivity;
- exhaustive cases are built with the inductive recursor or cases;
- recursive values in a countermodel are decoded only under an explicit depth bound or finite-domain argument.

Leant already serializes bounded constructor inventories and maps private engine names back to exact Lean constructors. The new source map should be shared with the SMT projection to prevent duplicate datatype identity mechanisms.

11.7 Arrays, maps, and extensionality

Arrays are more delicate because SMT arrays are total extensional functions with `select` and `store`, while Lean libraries expose several data structures and possibly different semantics. A safe first adapter should target a specific reflected array type and theorem set.

Reconstruction may use:

- read-over-write lemmas;
- simplification of equal and unequal indices;
- extensionality to reduce array equality to pointwise equality;
- finite-domain enumeration when the index type is finite;
- residual pointwise goals when full automation fails.

A Z3 model of an array often uses a default value plus finitely many stores. This can be rendered as an abstract diagnostic even when no exact Lean array term is available. Concrete counterexamples require an agreed array representation.

11.8 Polynomial identities and nonlinear arithmetic

For pure polynomial equalities, Z3 is not needed for the proof. It may help discover a normalization or parameter assignment; Lean should close the identity with a ring or Groebner-style tactic. For inequalities and nonlinear integer arithmetic, Z3 is incomplete and may return unknown. The system should use it for models and bounded templates while retaining ordinary Lean residual goals for unsupported proofs.

A useful hybrid is:

1. Z3 finds candidate coefficients or a counterexample;
2. Lean substitutes coefficients;
3. `grobner`, normalization, interval facts, or imported `mathlib` tactics discharge algebraic leaves;
4. any transcendental or genuinely nonlinear reasoning remains explicit.

11.9 Quantifiers

Quantifier reconstruction should never accept “Z3 instantiated something” as a proof step. Each instance must be traced to a source theorem or local hypothesis and rebuilt in Lean. The main services are:

- monomorphization proposals;
- ground term discovery;
- trigger and instance proposals;
- premise cores after instantiation;
- finite-domain enumeration where exact.

The reconstructed instances can then be consumed by `grind`. Unsupported higher-order or dependent instances are discarded. Leant's rank-N machinery is useful for deciding the structural placement of applications that an SMT solver sees only after monomorphization.

11.10 A reconstruction registry

Theory support should be registered by exact reflected operators and result strengths:

Listing 11.2: Schematic theory adapter registry

```
structure TheoryAdapter where
  name          : Name
  recognizes    : GoalSnapshot -> MetaM Recognition
  project       : GoalSnapshot -> Recognition -> MetaM SmtQuery
  validateModel : SmtModel -> MetaM ModelValidation
  reconstruct   : UnsatEvidence -> MetaM ReconstructionResult
  capabilities  : CapabilitySet
  version       : Nat
```

Adapters compose only through explicit shared terms. A query mixing unsupported operators is split into holes or rejected rather than silently approximated.

Cooperation with grind, TryThis, and premise selection

12.1 Three integration stages

A sensible progression avoids depending immediately on unstable internals.

12.1.1 Stage 1: sibling tactic portfolio

Each lane receives the same goal snapshot but operates independently:

- Leant returns a term or skeleton;
- Z3 returns a model/core/instance plan;
- Lean-native tactics attempt the goal directly;
- the broker reconstructs and ranks validated results.

Communication occurs through ordinary Lean facts and residual goals. This stage can be implemented entirely with public metaprogramming interfaces and TryThis.

12.1.2 Stage 2: fact exchange

The portfolio shares a normalized fact layer:

- grind preprocessing yields canonical expressions and relevant equivalence classes;
- Z3 receives only the facts and symbols in the selected dependency slice;
- reconstructed Z3 instances or arithmetic facts are inserted back as Lean theorems;
- Leant receives those facts as additional structural premises;
- bv_decide and other reflected solvers consume the same equalities.

This mirrors the current bv_decide integration without requiring Z3 to become a native grind solver.

12.1.3 Stage 3: solver extension

Only after an external extension API stabilizes should a Z3-backed component live inside GrindM. Such an extension might receive internalized atoms and equality updates, maintain an incremental solver, and inject checked conflict lemmas. It must still reconstruct every injected fact. The benefit is lower serialization overhead and tighter propagation; the cost is coupling to grind's backtracking, canonicalization, and proof-placeholder discipline.

12.2 How Leant can consume grind facts

Leant should not serialize the entire E-graph. It needs a selected view:

- canonical representatives for local terms used in the target or providers;
- proved equalities and disequalities with source proof references;
- simplified types and propositions;
- constructor facts and no-confusion consequences;
- selected arithmetic facts that have ordinary Lean proof expressions.

These facts can enter Djex as ordinary provider premises with exact names. A candidate that uses them is rendered back to the corresponding Lean proof reference. If a fact was only a transient internal inference with no reconstructed proof, it must not cross the boundary.

12.3 How Z3 can consume grind facts

Z3 benefits from the same canonicalization:

- fewer duplicate ground terms;
- smaller congruence problems;
- explicit constructor equalities;
- normalized arithmetic expressions;
- source-level theorem identities already attached to facts;
- better core explanations.

The projection should still be built from Lean expressions, not from untyped printed E-graph nodes. Every exported fact must have a Lean proof or be a source hypothesis.

12.4 Premise selection loop

A useful closed loop is:

1. LibrarySuggestions proposes a ranked premise prefix.
2. Leant and/or grind attempt the goal.
3. Z3 checks the first-order projection and returns a core or model.
4. Core members receive a temporary score boost; model-visible missing symbols trigger a focused retrieval expansion.
5. Reconstruction attempts the reduced problem.
6. The final accepted proof records actual theorem dependencies, which can be used as offline training or evaluation data.

No online score update should mutate global project state by default. The tactic may use ephemeral per-goal feedback; persistent learning should be an explicit tool with versioned data.

12.5 Suggestion ranking

Validated scripts can be ranked by a weighted tuple:

- closed proof before skeleton;
- fewer residual goals;
- fewer imported or nonlocal premises;
- smaller proof term or script;
- stable named theorems before fragile unfolding;
- native checked tactic before a large opaque generated term;
- fewer axiomatic dependencies under the chosen policy;
- lower measured replay time;
- engine-specific relevance score.

Ranking occurs after verification. A high-scoring raw candidate that does not elaborate has no standing.

12.6 A proposed `solve?` command

The portfolio command might behave as follows:

Listing 12.1: Illustrative portfolio command

```
example (n m : Nat) (h1 : n <= m) (h2 : m < n + 1) : n = m := by
  solve?
-- Candidate lanes:
--   omega: accepted
--   leant + omega skeleton: accepted, larger
--   z3 core -> omega: accepted, same premises h1 h2
-- Try this:
--   omega
```

The UI should show one default suggestion and make the audit comparison available on demand. The shortest accepted native tactic should normally win; Leant and Z3 add value on goals where native tactics need structural guidance, better premises, or semantic witnesses.

Quantifiers, higher-order structure, and theorem retrieval

13.1 The structural/semantic division is especially valuable here

SMT solvers operate most naturally on monomorphic first-order formulas. Lean goals frequently contain polymorphic declarations, higher-order functions, typeclass arguments, dependent binders, and proof terms. Leant/Djex is better positioned to construct the Curry–Howard application structure; Z3 is better positioned to choose useful ground instances once a safe first-order view exists.

A cooperative path is:

1. Leant identifies a hypothesis or provider whose result shape can solve the current structural goal.
2. The proof graph exposes typed argument holes, including type arguments and theorem premises.
3. Lean reifies supported first-order argument constraints.
4. Z3 proposes ground terms or a small set of instance assignments.
5. Lean elaborates the exact application and inserts the resulting proposition into grind.
6. Remaining higher-order or dependent arguments stay as explicit holes.

This avoids erasing higher-order structure merely to make the entire goal look like SMT-LIB.

13.2 Bounded rank-N planning

Leant’s current six-binder boundary is a deliberate finite search contract. Future work should improve *planning* before merely raising the bound:

- infer which quantified hypotheses can affect the result head;
- use type-argument unification to eliminate impossible instantiations;
- share repeated instantiation sites;
- let Z3 rule out inconsistent finite combinations of ground assignments;
- allocate the binder budget adaptively to the most relevant hypotheses;
- retain explicit truncation reasons when the frontier is incomplete.

The structural engine remains responsible for legal application and lambda placement. Z3 may choose among a finite set of typed assignments but should not invent untyped polymorphic terms.

13.3 Monomorphization as a proof-producing transformation

A monomorphizer should return more than a first-order formula. It should return:

- the source theorem and exact universe/type arguments;
- the instantiated Lean proposition;
- a proof expression obtained by applying the source theorem;
- the first-order encoding of that proposition;
- the dependency relation between generated instances;
- bounds and truncation status.

Then every SMT premise has an ordinary Lean proof before it reaches Z3. The solver can select or combine instances without adding trust.

The current `lean-smt` use of `lean-auto` for monomorphization is a relevant comparison. A Z3 integration can reuse or interoperate with such infrastructure rather than design a second incompatible instance language.

13.4 Model-guided instance generation

When Z3 returns a model for a quantified approximation, the model can suggest which terms distinguish the current candidate. A bounded loop is:

1. solve the current finite instance set;
2. inspect model values at trigger terms;
3. generate source-typed candidate instances;
4. validate and add them in Lean;
5. repeat under an explicit round and instance budget.

Failure to reach unsatisfiability is `Unknown` or `NoInstanceWithinBounds`, never a proof that the theorem is false. A concrete model can still be shown if it validates the finite approximation.

13.5 Theorem retrieval beyond symbol overlap

Premise selection can combine several signals:

- constants and type heads in the target;
- Leant provider-result compatibility;
- grind E-matching symbols and pattern availability;
- historical theorem dependencies;
- Z3 core frequency;
- model-visible missing relations;
- embeddings of normalized theorem statements;
- local declaration proximity and namespace/module preference.

The selector should return scored source declarations, not pretranslated SMT axioms. Lean decides whether and how each theorem is instantiated.

13.6 Abduction and missing-lemma suggestions

A useful proof-writing feature is to ask what additional proposition would make the current proof go through. General logical abduction is hard, but bounded forms are practical:

- choose a subset of candidate library lemmas using Boolean selectors;
- solve for a linear inequality template;
- infer an equality between existing terms;
- select a constructor invariant from a finite vocabulary;
- propose a missing precondition from a bounded atom set.

The result should be presented as a conjectured helper lemma or missing assumption. Lean must prove it separately. This can turn a failed automated proof into a useful next editing action rather than a generic timeout.

Induction, recursion, invariants, and termination

14.1 Why induction is the next major frontier

Most current structural and SMT automation is strongest on local, nonrecursive goals. Real Lean developments frequently fail at choosing:

- the induction variable or recursor;
- a generalized motive;
- which variables must be reverted;
- an auxiliary invariant;
- a termination measure;
- a strengthening of the statement that is stable under recursion.

These choices are not well served by one engine. Leant can synthesize recursor and constructor structure; Z3 can solve semantic templates and find counterexamples to weak invariants; Lean can generate and check the exact induction obligations.

14.2 Induction-skeleton synthesis

Extend the proof graph with an induction node that records:

- the major premise or recursive argument;
- the exact recursor or induction theorem;
- the proposed motive;
- generalized local variables;
- branch contexts and induction hypotheses;
- residual branch goals;
- termination or well-foundedness obligations where applicable.

Leant can search a bounded set of structurally plausible induction plans. Plans are reconstructed immediately in Lean; ill-typed motives or recursor applications are rejected before semantic solving.

14.3 Invariant synthesis loop

For a loop, recursive function, state machine, or inductive proof, use a template-checking loop:

1. Lean chooses the exact state variables and generates verification conditions: initialization, preservation, exit/postcondition, and optionally decrease.
2. A bounded template language is built from source terms and relations.
3. Z3 finds coefficients or selectors satisfying a finite encoding of the VCs, or returns a countermodel to the current template.
4. The candidate invariant is reconstructed as a Lean predicate.
5. Lean attempts all VCs with native tactics and the proof portfolio.
6. A failed VC yields a checked counterexample or a stronger template request.
7. The process terminates under explicit rounds, coefficient bounds, and template growth limits.

The final theorem depends only on the Lean-checked invariant proof. Z3 is a generator.

14.4 Linear and lexicographic ranking functions

Termination measures are well suited to SMT template solving. For a transition $x \rightarrow x'$, search for coefficients a_i such that

$$\rho(x) = a_0 + \sum_i a_i x_i, \quad \rho(x) \geq 0, \quad \rho(x') < \rho(x).$$

For lexicographic tuples, add selector variables identifying the first decreasing component and prove nonincrease of earlier components. Lean receives the concrete measure and proves the well-founded decrease obligation. Recent Lean work on well-founded and state-dependent measures in verification-condition generation makes this direction particularly compatible with the native ecosystem.

14.5 Predicate discovery from cores and models

A template vocabulary can grow from failed attempts:

- unsat cores identify predicates essential to preservation;
- models expose state components or relations not constrained by the invariant;
- constructor branches contribute discriminator predicates;
- array models contribute selected indices;
- theorem retrieval contributes named domain lemmas;
- Leant contributes structurally available projections and recursive fields.

This is a proof-oriented form of counterexample-guided abstraction refinement. Every new predicate has a source identity and exact Lean type.

14.6 Recursive data and fold lemmas

For recursive inductives, Leant already distinguishes complete and partial constructor inventories and uses different projections for Djinn and Exference. Future proof automation can add *family descriptors*:

- exact inductive head and parameters;
- constructor fields and recursive positions;
- available recursors, folds, maps, and eliminators;
- algebraic laws and homomorphism lemmas registered for `grind`;
- size or measure functions;
- finite-model bounds for counterexample search.

A family descriptor lets Leant synthesize a fold/recursor skeleton while Z3 reasons about an abstract measure or state relation. The exact recursive proof remains in Lean.

14.7 Statement strengthening

When an induction hypothesis is too weak, bounded strengthening can search a grammar of candidate conjuncts or generalized variables. Z3 can reject candidates with concrete countermodels to the step condition. Lean verifies the surviving strengthened theorem.

The UI should show the proposed stronger lemma explicitly. Automatic strengthening must not silently change the theorem being proved; it should insert a local helper or suggest a new declaration.

Counterexample-guided synthesis and proof repair

15.1 CEGIS for proof terms and witnesses

Leant/Djex can generate structural candidate families; Z3 can distinguish them with models. A bounded counterexample-guided loop is:

1. Leant generates a typed candidate or sketch.
2. Lean elaborates it and extracts supported behavioral obligations.
3. Z3 searches for a counterexample to those obligations.
4. A validated counterexample becomes a new test constraint on the search.
5. Leant/Exference generates the next candidate under accumulated constraints.
6. When no counterexample is found, Lean still proves the final obligation; the absence of a Z3 model is not accepted as the theorem unless reconstructed.

This is useful for constructive witnesses, finite functions, data-structure transformations, and proof terms with finite semantic choices. The type system and structural search guarantee that every candidate is well-formed; semantic constraints focus the enumeration.

15.2 Semantic fingerprints and candidate equivalence

Many structurally different candidates behave identically on the current finite model set. The broker can assign a semantic fingerprint: the vector of evaluated outputs or predicates over validated counterexamples. Exference can keep the cheapest candidate per fingerprint and avoid spending Lean verification budget on equivalent variants.

This is a heuristic quotient only. Two candidates with the same finite fingerprint are not definitionally or extensionally equal in Lean. The final selected candidate is proved independently.

15.3 Proof repair after source changes

A broken proof often retains most of its structure. Store an optional proof graph and audit trail beside generated scripts. On failure after a refactor:

1. re-resolve source declarations by stable names and type fingerprints;
2. reconstruct the unchanged prefix of the graph;
3. turn failed nodes into typed holes;
4. use theorem retrieval to find replacement declarations;

5. ask Z3 for a core or counterexample explaining changed semantic assumptions;
6. ask Leant to resynthesize only the affected structural region;
7. render a new validated script and show the dependency changes.

This is more robust than rerunning a monolithic tactic from scratch and more maintainable than storing an opaque proof term.

15.4 Diagnosing false or underspecified lemmas

When a goal appears unprovable, the portfolio can produce a structured diagnostic:

- Leant reports whether the structural fragment is refuted, incomplete, or outside bounds;
- Z3 reports a validated or abstract model of the first-order projection;
- the premise selector reports potentially missing relevant theorems;
- an abduction service proposes bounded missing assumptions;
- projection reports identify definitions or dependent structure that prevented a strong conclusion.

This is often more valuable to a proof author than another tactic timeout.

15.5 Synthesis under behavioral constraints

The same architecture can synthesize a term and its proof together. For example, a candidate function may be required to preserve list length, maintain a finite-state invariant, or satisfy a postcondition. Leant builds the term skeleton; Z3 filters semantic choices and proposes invariants; Lean proves the final specification. This extends proof automation into verified program synthesis without changing the trust model.

15.6 Limits of finite counterexamples

Finite testing and bounded models are powerful search tools but provide no completeness for general Lean propositions. The protocol must record:

- domain bounds and enumeration strategy;
- whether recursive values were depth-bounded;
- whether functions were represented extensionally or as uninterpreted symbols;
- whether arithmetic was mathematical, bit-vector, or bounded;
- which quantifiers were instantiated rather than universally checked.

The UI should say “no counterexample within bounds” rather than “proved”.

Concrete protocol and API design

16.1 Design goals

The protocol must support independent evolution of Lean, Leant/Djex, and Z3 while making semantic drift obvious. Its goals are:

- no dependence on pretty-printed Lean syntax for identity;
- explicit types, binders, universes, and source references;
- bounded parsing and memory use;
- streaming candidates and progress;
- versioned capabilities;
- deterministic recording and replay;
- forward-compatible unknown fields;
- strong rejection of malformed, stale, or inconsistent data;
- separation of semantic payload from user-facing rendering.

16.2 Stable identities

The Lean side assigns compact IDs within one snapshot:

Listing 16.1: Schematic identity types

```

structure SnapshotId where value : UInt64
structure SourceId   where value : UInt32
structure BinderId  where value : UInt32
structure TermId     where value : UInt32
structure TypeId     where value : UInt32
structure FactId     where value : UInt32
structure NodeId     where value : UInt32
structure HoleId     where value : UInt32

```

IDs are never reused within a snapshot. A response carrying an unknown ID is rejected. A global declaration additionally carries its fully qualified Name, universe arity, and a type fingerprint. The fingerprint protects against resolving the same name to a changed declaration after recompilation.

16.3 Semantic term and type AST

The interchange AST should be smaller than Lean's full Expr but richer than Leant's current rendered atoms. It may include:

- sorts and universe parameters represented symbolically;

- local and global references;
- applications with explicit argument visibility;
- nondependent and dependent function types;
- lambdas and lets where needed for proof graphs;
- products, sums, propositions, equality, and selected inductive applications;
- literals for supported arithmetic and bit-vector domains;
- opaque nodes with exact source IDs and inferred types;
- no arbitrary embedded tactic syntax.

Every node has a type ID. The parser checks graph acyclicity where required, index bounds, arities, and sort consistency. Lean nevertheless rechecks the reconstructed expression; protocol type checking is a robustness and diagnostic feature, not the final trust boundary.

16.4 Binder visibility and implicit information

Lean elaboration depends on binder information. The protocol must distinguish:

- explicit, implicit, strict implicit, and instance-implicit binders;
- term binders from type/sort binders;
- source-visible names from anonymous/internal names;
- binders that Leant erases semantically but must recreate in a lambda;
- exact provider type arguments inferred from instance heads;
- whether an argument should be emitted explicitly or left for elaboration.

Current Leant already records explicit forall visibility, rendering slots for instance binders, exact provider binder names, kind arities, and semantic forall-domain tags. The typed protocol should generalize those successful mechanisms rather than replace them with raw source strings.

16.5 Projection capabilities

The request includes a capability declaration. For example:

Listing 16.2: Illustrative capability negotiation

```
{
  "capabilities": {
    "proofGraph": 2,
    "residualHoles": true,
    "rankNBinders": 6,
    "exactProviderAssignments": true,
    "structuralProdSumKinds": true,
    "smt": ["QF_UF", "QF_LIA", "QF_BV", "QF_AUFBV"],
    "modelValidation": ["finite", "bitvector", "integer"],
    "evidence": ["candidate", "core", "model", "instancePlan"]
  }
}
```

A worker must not infer permission from a schema version. Unsupported requested capabilities produce a structured refusal before search starts.

16.6 Progress and truncation

Progress is a first-class stream:

Listing 16.3: Schematic progress report

```
inductive TruncationReason
| timeout
| heartbeatLimit
| memoryLimit
| candidateLimit
| choicePointLimit
| queuePruned (count : Nat)
| binderLimit (observed supported : Nat)
| providerLimit
| instanceLimit
| depthLimit
| solverUnknown (reason : String)
| interrupted

structure ProgressReport where
  phase      : Name
  explored   : Nat
  candidates : Nat
  elapsedMs  : Nat
  truncation : Array TruncationReason
  completeFor : Array Capability
```

The phrase `completeFor` is important. A search may be complete for the pure structural core while incomplete for provider widening or quantifier instantiation.

16.7 Evidence payloads

A common evidence type can be:

Listing 16.4: Schematic evidence vocabulary

```
inductive ExternalEvidence
| candidateOnly
| structuralDerivation (derivation : DerivationGraph)
| structuralRefutation (refutation : RefutationGraph)
  (adequacy : NegativeAdequacyLevel)
| model (model : SmtModel) (validation : ModelValidationLevel)
| unsatCore (facts : Array FactId) (minimality : CoreMinimality)
| instantiations (plans : Array InstantiationPlan)
| theoryPlan (plan : TheoryProofPlan)
| certificate (format : CertificateFormat) (bytes : ByteArray)
| proofTrace (format : ProofTraceFormat) (bytes : ByteArray)
| invariantTemplate (template : TemplateAssignment)
| noEvidence
```

Only certificate in a registered checked format and a successfully reconstructed proof can become goal-closing evidence. `proofTrace` is deliberately weaker.

16.8 Candidate rejection taxonomy

Rejected candidates should be classified rather than silently skipped:

- stale snapshot;
- unknown or changed declaration;
- malformed graph or out-of-range ID;
- type mismatch at a node;
- unresolved universe constraint;
- failed typeclass synthesis;
- unsupported coercion or elaboration ambiguity;
- unresolved non-hole metavariable;
- kernel rejection;
- timeout or allocation limit during verification;
- residual hole outside the advertised capability;
- forbidden axiom/unsafe dependency under policy;
- duplicate of an earlier semantic candidate.

These categories are invaluable for improving the renderer and protocol. A raw “candidate failed” message hides whether the search or the bridge is responsible.

16.9 Schematic Lean reconstruction API

Listing 16.5: Possible Lean-side API surface

```
namespace ProofPortfolio

structure ReconstructConfig where
  allowResidualGoals : Bool := true
  maxResidualGoals   : Nat := 8
  verifyKernel       : Bool := true
  axiomPolicy        : AxiomPolicy := .report

opaque snapshotGoal : MVarId -> MetaM GoalSnapshot
opaque projectForLeant : GoalSnapshot -> MetaM LeantRequest
opaque projectForSmt   : GoalSnapshot -> TheoryAdapter -> MetaM SmtQuery

opaque reconstructGraph :
  GoalSnapshot -> ProofGraph -> ReconstructConfig -> MetaM ReconstructionResult

opaque validateModel :
  GoalSnapshot -> SmtQuery -> SmtModel -> MetaM ModelValidation
```

```

opaque replayTheoryPlan :
  GoalSnapshot -> TheoryProofPlan -> MetaM ReconstructionResult

opaque addTryThisSuggestion :
  GoalSnapshot -> ReconstructionResult -> TacticM Unit

end ProofPortfolio

```

The opaque declarations above indicate module boundaries, not axioms in the final implementation.

16.10 Schematic Haskell worker API

Listing 16.6: Possible Leant/Djex worker boundary

```

data SearchRequest = SearchRequest
  { requestSnapshot    :: SnapshotId
  , requestGoal        :: TypedFragment
  , requestProviders   :: [Provider]
  , requestPremises    :: [Premise]
  , requestCapabilities :: CapabilitySet
  , requestLimits      :: SearchLimits
  }

data SearchEvent
  = CandidateEvent CandidateGraph Provenance RankingData
  | DiagnosticEvent Diagnostic
  | ProgressEvent ProgressReport
  | FinishedEvent Completion

runSearch :: SearchRequest -> ConduitT () SearchEvent IO ()

```

Internally, pure Djinn and Exference search can remain pure. The outer driver handles streaming, time-outs, semantic callbacks, and cancellation.

Scheduling, caching, and process isolation

17.1 A deterministic portfolio scheduler

Parallelism is useful, but proof automation must remain reproducible enough to debug. The scheduler should define a deterministic priority and budget policy even when lanes run concurrently.

A reasonable default sequence is:

1. cheap simplification, assumption, constructor, and native exact tactics;
2. capability-specific native solver;
3. Leant Djinn small structural search;
4. premise selection and grind;
5. Leant combined structural search;
6. Z3 counterexample/core/instance service;
7. reconstruction with reduced premises;
8. larger Exference or invariant/template search;
9. return the best validated result available under the total budget.

Lanes may overlap in wall-clock execution, but candidate ranking and tie-breaking use stable lane priorities, semantic cost, and candidate IDs. A faster nondeterministic arrival should not permanently change the default proof script when two accepted results are equivalent in cost.

17.2 Cancellation

Cancellation must propagate through:

- editor/tactic interruption;
- Lean meta computation;
- Leant worker request;
- pure search forcing;
- Z3 process or API interrupt;
- candidate verification;
- pending cache writes.

Every request has a cancellation token and deadline. A cancelled worker is not reused unless its protocol state is known to be clean. If a Z3 incremental session cannot be safely unwound after interruption, terminate and replace the process.

17.3 Cache layers

Useful caches include:

Projection cache

Normalized structural and SMT projections keyed by the goal snapshot and adapter version.

Provider cache

Leant's existing environment/session replay cache, keyed by import history and exact provider query.

Candidate cache

Proof graphs from Leant/Djex keyed by typed fragment, provider inventory, limits, and engine revision.

Solver cache

Z3 model/core/template results keyed by canonical SMT query, solver revision, strategy, and options.

Hole-proof cache

Kernel-accepted proofs for normalized residual goals under a conservative context dependency slice.

Rendering cache

Validated source scripts keyed by proof graph and pretty-printer/options version.

A cache hit never bypasses verification. Cached proof graphs are reconstructed against the current snapshot; cached scripts are re-elaborated; cached models are revalidated if used as current diagnostics.

17.4 Incremental sessions

Leant already has a pure replay plan distinguishing reuse, suffix replay, and full replay when the interactive command history changes. The native service should expose this mechanism explicitly. Similarly, a Z3 session can retain stable declarations and use push/pop for goal-specific assertions.

Incrementality must not leak semantic context:

- every declaration belongs to an environment generation;
- every goal runs at a known solver stack depth;
- cancellation restores or destroys the session;
- proof/model/core options are fixed per worker class;
- random seeds and tactics are recorded;
- assertions are named with snapshot-scoped IDs.

17.5 Resource isolation

External workers should run with:

- memory and CPU limits;
- bounded stdout/stderr capture;
- no network access by default;
- a controlled working directory;
- explicit executable paths and hashes;
- no ability to modify source files;
- sanitized environment variables;
- process replacement after protocol errors or excessive memory growth.

This is ordinary robustness, not an implication that the tools are malicious. Search engines routinely encounter pathological inputs.

17.6 Deterministic replay bundles

Every nontrivial failure should be exportable as a small replay bundle containing:

- the semantic request with source text redacted optionally;
- exact executable revisions and options;
- provider/premise inventory fingerprints;
- worker and solver event stream;
- candidate graph or proof trace;
- reconstruction error and Lean messages;
- random seeds and resource limits.

Replay bundles make cross-platform solver differences and version regressions tractable. They also permit protocol fuzzing without a live Lean project.

User experience and proof presentation

18.1 Commands and their promises

A coherent command family should make result strength visible:

Command	Behavior	Promise
leant?	Show validated structural term or skeleton suggestions.	Every shown closed term elaborates at the current goal; skeletons list residual goals.
leant	Apply the best validated Leant result.	Changes proof state only through accepted Lean expressions.
z3?	Show a reconstructed proof suggestion, core, or model depending on capability.	Output labels distinguish proof, concrete counterexample, abstract model, core, and unknown.
counterexample?	Search exact and bounded model adapters.	Never claims a theorem; reports realization and bounds.
solve?	Run the proof-producing portfolio and show the best validated script.	Only checked proof lanes are offered as goal-closing actions.
explain_failure	Summarize structural bounds, projection loss, models, cores, and rejected candidates.	Diagnostic only.

18.2 Result wording

The UI should use exact language. Examples:

- “Proof accepted by Lean.”
- “Proof skeleton accepted; 2 residual goals remain.”
- “Concrete counterexample validated for $x = 0, y = 1$.”
- “Abstract first-order model found; uninterpreted element $U!0$ has no concrete Lean decoder.”
- “Z3 returned an unsat core containing 4 of 19 translated premises; Lean reconstruction succeeded using 3 premises.”
- “No Leant candidate found within the six-binder and 60-candidate bounds.”
- “Djinn refuted the exact pure structural projection.”
- “Solver returned unknown: incomplete quantified reasoning.”

Phrases such as “Z3 proved” should be reserved for a result that was actually reconstructed or certificate-checked in Lean.

18.3 Readable proof selection

The portfolio should offer modes:

Compact

Prefer the shortest accepted term or tactic.

Readable

Prefer named intermediate facts, explicit cases, and stable theorem applications.

Robust

Prefer proofs with fewer unfolding dependencies, smaller premise sets, and successful replay across normalization variants.

Diagnostic

Preserve solver provenance, cores, and comments in a generated proof draft.

The generated source can include comments only in diagnostic mode; the semantic proof graph remains comment-free.

18.4 Interactive steering

Users should be able to constrain the portfolio without writing a new tactic:

Listing 18.1: Illustrative steering options

```
by
  solve? (using := [h1, h2, theoremName])
        (without := [expensiveLemma])
        (engines := [.grind, .leant, .z3])
        (maxPremises := 24)
        (allowInduction := false)
        (counterexamples := .finite 8)
```

These options influence search only. The final proof remains ordinary Lean.

18.5 Explanations from proof graphs

A proof graph can generate a concise explanation:

Introduce x . Apply `map_preserves` to the local hypothesis h ; this creates a structural equality and a numeric bound. The equality is solved by simplification. Z3 selected hypotheses `hBound` and `hNonneg` as an unsatisfiable arithmetic core; `omega` reconstructed the bound. The resulting term was accepted by Lean.

This is more reliable than asking an external solver to narrate an opaque proof trace. The explanation is generated from checked nodes and source identities.

18.6 Proof drafts and helper lemmas

When a goal is not closed, the tactic can still emit a useful draft:

Listing 18.2: Illustrative partial proof draft

```
by
  induction xs with
  | nil => simp
  | cons x xs ih =>
    refine constructor
    . exact ?length_bound
    . simpa using ih
```

The editor can focus the first residual hole. A separate action may insert a proposed helper lemma or invariant statement above the theorem, but this must be explicit because it changes the source-level proof plan.

Implementation roadmap and acceptance gates

19.1 Principle of the roadmap

Each milestone should end with a useful, independently testable feature. Later stages must not be prerequisites for trusting earlier ones. In particular, Z3 proof-object support is not required for a valuable counterexample/core/instance service.

19.2 Milestone 0: shared fixtures and revision discipline

Deliverables

- Pin exact Lean, Leant, Djex, and Z3 revisions in integration tests.
- Extract current Leant serializer, provider, candidate, and transcript fixtures.
- Define snapshot IDs, source IDs, progress, truncation, and result-strength constructors.
- Build protocol fuzz tests and malformed-input limits.
- Establish a benchmark corpus spanning propositional structure, providers, rank-N goals, arithmetic leaves, bit vectors, datatypes, and negative cases.

Acceptance gate: all existing Leant transcript candidates are replayable from recorded fixtures; malformed and stale messages fail closed; benchmark results record exact revisions and limits.

19.3 Milestone 1: native leant? using current candidate strings

Deliverables

- Lean tactic package and long-lived Leant worker.
- Goal snapshot, current semantic fragment request, provider query, cancellation, and streaming progress.
- Candidate verification in the exact saved tactic state.
- TryThis suggestions and a detailed rejection trace.
- Compatibility tests over supported Lean versions.

Acceptance gate: every displayed candidate elaborates and kernel-checks at the current goal; editor cancellation leaves no worker/session corruption; existing live Leant examples work without manually replaying them in a separate REPL.

19.4 Milestone 2: typed proof graphs and residual holes

Deliverables

- Versioned semantic term/type AST and stable IDs.
- Djex/Leant typed candidate projection.
- Lean reconstruction to Expr and refine skeletons.
- Hole capability analysis and native tactic dispatch.
- Graph sharing, semantic deduplication, and readable rendering.

Acceptance gate: candidates that formerly failed because of one arithmetic or simplification leaf now produce accepted skeletons; no embedded worker source text is required for semantic identity; all holes have Lean-checked expected types.

19.5 Milestone 3: Z3 counterexamples and unsat cores

Deliverables

- Typed SMT AST and process pool.
- Exact QF_UF, QF_LIA, QF_BV, and selected finite-datatype adapters.
- Independent model parser/evaluator and concrete/abstract model labels.
- Fast and deletion-minimized assumption cores.
- Source mapping and editor diagnostics.

Acceptance gate: every reported concrete model is independently re-evaluated; unknown is preserved; models after unknown are never labeled counterexamples; core-guided premise reduction demonstrably reduces replay problems without changing trust.

19.6 Milestone 4: theory-specific reconstruction

Deliverables

- EUF equality-plan reconstruction.
- Core-to-grind and core-to-arithmetic replay.
- Z3-guided omega/linear arithmetic and bv_decide dispatch.
- Finite datatype constructor/case reconstruction.
- Residual-goal fallback for unsupported array and quantified steps.

Acceptance gate: a Z3 unsat closes a goal only after the reconstructed Lean proof succeeds; disabling Z3 and replaying the generated script still proves the theorem.

19.7 Milestone 5: cooperative solve? and premise feedback

Deliverables

- Deterministic portfolio scheduler.
- Shared normalized fact/source layer with grind.
- LibrarySuggestions adapter for Leant providers and Z3 feedback.
- Candidate ranking modes and proof audit reports.
- Cache and replay bundle infrastructure.

Acceptance gate: the portfolio never returns a weaker diagnostic when a checked proof is available; result ordering is stable under repeated runs with fixed options; shared-state paths outperform independent tactic racing on selected mixed benchmarks.

19.8 Milestone 6: induction and invariant synthesis

Deliverables

- Induction-skeleton proof nodes and bounded motive/generalization search.
- Linear and lexicographic invariant/ranking templates.
- Z3 template and fixedpoint adapters.
- Lean VC generation and recursive portfolio solving.
- Explicit helper-lemma/strengthening suggestions.

Acceptance gate: every synthesized invariant and termination measure is materialized as Lean syntax and all VCs are kernel-checked; failed or bounded searches produce counterexamples/unknown labels rather than false success.

19.9 Milestone 7: selective certificate and proof-trace replay

Deliverables

- Identify a stable subset of Z3 proof rules or a translation to a standard certificate format.
- Implement a bounded parser and small Lean checker for that subset.
- Fall back to theory reconstruction or residual holes for unsupported rules.
- Differentially test checker results against native reconstruction and solver self-validation.

Acceptance gate: the checker has an explicit Lean soundness theorem; no unsupported proof rule is accepted; solver upgrades cannot silently change the rule semantics.

19.10 Recommended stopping points

Milestones 1 through 4 already form a highly useful system. Milestone 5 adds integrated polish and performance. Milestone 6 is a research-rich extension. Milestone 7 should be undertaken only if profiling shows that native reconstruction leaves a significant and valuable class of goals unsolved.

Evaluation methodology

20.1 Metrics

The evaluation should separate search quality, reconstruction quality, proof quality, and diagnostics.

Metric	Meaning
Kernel acceptance rate	Fraction of generated closed candidates accepted at the exact target.
Skeleton utility rate	Fraction of skeletons for which native/Z3-guided lanes close at least one residual goal, and fraction fully closed.
Time to first accepted proof	End-to-end latency including projection, external search, reconstruction, and checking.
Readable-script rate	Accepted results whose rendered script meets size and stability criteria.
Candidate rejection distribution	Bridge defects versus genuine search mismatch: names, universes, instances, type mismatch, kernel rejection, resource limit.
Premise reduction	Initial suggestions versus Z3 core versus actual dependencies of the accepted proof.
Model validation rate	Raw solver models versus independently validated concrete or abstract models.
Unknown/truncation rate	Frequency and reasons, grouped by theory and goal shape.
Search work	Djinn choice points, Exference nodes/queue pruning, Z3 conflicts/rlimit, generated instances, candidates verified.
Residual-goal complexity	Count and capability class before and after portfolio routing.
Replay stability	Success under repeated runs, theorem reordering, alpha-renaming, import changes, and supported Lean/Z3 upgrades.
Cache effectiveness	Hit rate, revalidation cost, stale rejection rate, and memory footprint.

20.2 Benchmark strata

The corpus should contain:

1. pure propositional and Curry–Howard combinator goals;
2. products, sums, equivalence, negation, and finite inductives;
3. local and library provider compositions;
4. explicit/implicit/rank-N binders through and beyond the supported boundary;

5. exact active-instance provider assignments, including higher-kinded cases;
6. structural skeletons with one or more arithmetic leaves;
7. EUF and quantified theorem-instantiation problems;
8. bit-vector verification goals;
9. finite datatype and array examples;
10. false statements with concrete and abstract models;
11. goals requiring induction, invariants, or strengthened hypotheses;
12. adversarial malformed protocol inputs and resource bombs;
13. real proofs from Lean core, Std, and selected external projects, respecting licensing and dependency constraints.

20.3 Baselines

Compare against:

- `simp`, `grind`, `omega`/arithmetic tactics, `grobner`, and `bv_decide` alone;
- current Leant REPL behavior at the pinned functional revision;
- Leant native tactic without Z3;
- Z3 diagnostic/core service without Leant;
- independent tactic racing versus shared facts and cores;
- `lean-smt` on its supported `cvc5` theories where comparison is meaningful;
- explicit human-written proofs for readability and replay stability.

The objective is not to beat every specialized tactic. The portfolio should invoke the specialized tactic when it is already the best answer.

20.4 Metamorphic tests

Automation should preserve behavior under transformations that should not affect the proof problem:

- alpha-renaming local hypotheses and binders;
- reordering independent hypotheses;
- adding irrelevant hypotheses and declarations;
- changing user-facing universe parameter names;
- replacing a term by a definitionally equal form;
- reassociating products/sums where explicit equivalences are supplied;
- changing theorem declaration order while retaining names and types;
- replaying with a cold versus warm worker cache;

- serializing and reparsing the same protocol request;
- varying solver seeds where the result strength should remain the same.

Candidate ranking may change under some transformations, but logical labels and final kernel acceptance must not.

20.5 Differential tests

Useful differential checks include:

- Z3 model evaluation versus a separate evaluator and Lean computation;
- Z3 core replay versus native tactics on the selected facts;
- structured Leant reconstruction versus current rendered-string verification;
- grind with and without Z3-proposed instances;
- bit-vector counterexamples versus `bv_decide`;
- proof-graph rendering versus direct `Expr` reconstruction;
- any future proof-trace checker versus solver self-validation and native reconstruction.

Disagreement is a test failure or an explicit unsupported case, never silently resolved in favor of the external solver.

20.6 Adversarial protocol tests

Fuzz and hand-write cases with:

- cycles in supposedly acyclic graphs;
- invalid node IDs and duplicate IDs;
- wrong arities, sorts, and binder counts;
- huge integers, strings, arrays, and nesting depth;
- malicious source names and Unicode edge cases;
- stale snapshot IDs and changed declaration fingerprints;
- candidates with unresolved metavariables or hidden sorry;
- solver output after cancellation;
- incomplete JSON/binary frames;
- inconsistent progress/completion claims;
- proof certificates exceeding declared limits.

The expected outcome is a bounded structured error and an unchanged proof state.

Risks, semantic traps, and non-goals

21.1 Semantic mismatch risks

21.1.1 Natural numbers versus integers

SMT integer subtraction is total and may be negative; Lean natural subtraction is truncated. Division and modulus conventions also need exact agreement. Projections must use side conditions or dedicated operators rather than rely on visual similarity.

21.1.2 Partial operations

Field inversion, array indexing wrappers, head/tail operations, and user definitions may have defaults or require hypotheses. A translator must include the source semantics and side conditions or leave the term opaque.

21.1.3 Prop versus Bool

Lean propositions and Boolean computations are related by reflection lemmas, not identical. Every conversion must be explicit and reconstructed.

21.1.4 Functions and extensionality

SMT functions and arrays are extensional. Lean function equality requires function extensionality, available as a theorem/axiom in the environment. The proof audit should report such dependencies. Models of higher-order functions are generally abstract.

21.1.5 Datatypes and recursion

Z3 algebraic datatypes have first-order semantics; Lean inductives may be indexed, dependent, nested, quotient-based, or involve proof fields. Only exact admitted families should be translated structurally. Recursive countermodels need finite realization or explicit bounds.

21.1.6 Type classes and coercions

Typeclass synthesis can choose evidence based on the current environment and local instances. Lean's exact provider-assignment discipline should be retained. Z3 should never model class dictionaries as interchangeable opaque values when their identity affects elaboration or computation.

21.1.7 Definitional equality and transparency

An SMT projection sees explicit equations; Lean also has definitional reduction under configurable transparency. Projection and reconstruction should normalize through a named, recorded policy. A proof that depends on aggressive unfolding may be fragile and should rank below a stable theorem-based proof when possible.

21.1.8 Quotients, proof irrelevance, and subsingletons

These features can invalidate naive first-order encodings. Treat them as opaque until a dedicated adapter provides exact theorems.

21.2 Engineering risks

Risk	Failure mode	Mitigation
Lean API drift	Projection/reconstruction package breaks across releases.	Thin version adapter, protocol fixtures, support matrix, avoid private grind internals initially.
Solver drift	Different models, cores, traces, performance, or proof rules.	Pin revisions/options, record scripts, validate results independently, replace workers cleanly.
Search explosion	Combined structural, premise, instance, and semantic search becomes unusable.	Capability routing, explicit budgets, staged widening, core reduction, DAG sharing, deterministic truncation.
Poor proof readability	Accepted terms are enormous or fragile.	Post-verification ranking, proof-graph rendering modes, native tactic preference, helper lemmas.
Protocol complexity	Bridge becomes harder to maintain than the engines.	Small semantic AST, capability negotiation, generated codecs, bounded schema, replay fixtures.
False confidence from models	Abstract/bounded model shown as a concrete refutation.	Result lattice, realization labels, independent evaluator, exact wording.
Unsound pruning	Incomplete projection deletes a valid synthesis branch.	Exact-versus-heuristic pruning policy; unverified results change ranking only.
Resource leakage	Cancelled incremental sessions corrupt later queries.	Strict worker state machine, generation IDs, process replacement after uncertain state.
Axiom surprises	Kernel-accepted proof depends on unexpected axioms.	Optional axiom-policy audit and dependency display.
Licensing/distribution	Bundling external solver complicates packaging.	Support configured external executable first; document versions and licenses; optional managed download separately.

21.3 Non-goals for the initial system

The first implementation should not attempt:

- complete automation for arbitrary dependent type theory;
- proving global non-inhabitation in an unrestricted Lean environment;
- translating arbitrary Lean computation into SMT;
- trusting raw solver unsat;
- universal Z3 proof replay;
- unrestricted higher-order or impredicative model finding;
- automatic insertion of unproved helper lemmas or assumptions;
- replacing specialized Lean tactics that already solve the goal well;
- hiding resource bounds or incomplete search behind a binary failure result.

21.4 What success looks like

A successful system does not need to solve every theorem. It should reliably turn many currently awkward proof states into one of four useful outcomes:

1. a short kernel-accepted proof;
2. a well-typed structural skeleton with focused residual goals;
3. a validated counterexample or precise abstract model;
4. an actionable explanation: minimal premises, missing instance, exceeded bound, unsupported projection, or solver unknown reason.

Prioritized recommendations

22.1 Recommendation 1: implement leant? before changing the search

This is the clearest path from the current repositories to immediate proof-authoring value. The existing serializer, engine, renderer, verification boundary, provider logic, rank-N tests, and progress semantics can all be reused. The work establishes the native Lean package, worker lifecycle, versioning, cancellation, and suggestion UX required by every subsequent direction.

22.2 Recommendation 2: make typed holes the next core abstraction

Typed holes unlock cooperation. Without them, every unsupported arithmetic or domain leaf discards the whole structural candidate. The proof-graph protocol should be small, source-mapped, and reconstructable into refine. This is a higher-leverage change than increasing search bounds.

22.3 Recommendation 3: add Z3 first as a model/core service

Models and cores give excellent diagnostics and search guidance without a large proof checker. Implement exact QF_LIA/QF_BV/finite-datatype model validation and assumption-based cores. Feed cores into grind and Leant provider selection. Preserve unknown and distinguish concrete from abstract models.

22.4 Recommendation 4: reconstruct with native theory tactics

For EUF, linear arithmetic, bit vectors, and datatypes, use Z3 to reduce and explain the problem, then let Lean prove it. This obtains proof-producing SMT assistance without coupling to Z3's full proof language.

22.5 Recommendation 5: integrate through public facts before private solver state

Follow the successful bv_decide/grind direction, but in stages. First exchange proved Lean facts and normalized source expressions. Design a genuine grind external-solver extension only after concrete workloads reveal the benefit and a stable API can be justified.

22.6 Recommendation 6: preserve evidence/progress distinctions end to end

Leant already distinguishes candidates, refutation, bounded failure, and truncation. Z3 distinguishes sat, unsat, and unknown. The common protocol must preserve these distinctions and add reconstruction/model-validation status. This is a correctness feature, not merely better logging.

22.7 Recommendation 7: treat induction as a proof-plan problem

Do not ask Z3 to prove arbitrary recursive Lean theorems directly. Let Leant generate recursor/motive skeletons and let Z3 solve bounded invariant or ranking templates. Lean checks all verification conditions. This division makes ambitious automation possible without widening the trusted base.

22.8 Recommendation 8: defer universal proof-object checking

A broad Z3 proof checker may eventually be valuable, especially for performance or proof portability. It should follow working theory-specific reconstruction and be justified by measured gaps. Start with compact, stable certificate formats and explicitly unsupported residual rules.

22.9 Recommended first prototype in one page

Prototype specification

Name: `Leant.Tactic`

Input: active `MVarId`, current environment, tactic options.

Worker: revision-pinned long-lived Leant process using current Djex submodule, current structural fragment, and bounded provider discovery.

Output: streaming candidate source strings plus structured progress and projection diagnostics.

Verification: restore saved tactic state; elaborate candidate at exact target; reject unresolved metavariables or sorry; run normal kernel checking.

UI: `leant?` emits a validated `TryThis` suggestion; `leant` applies it; `leant_trace` explains failures and bounds.

Tests: existing Leant live transcripts, alpha-renaming, hypothesis reordering, stale snapshots, cancellation, malformed worker output, candidate rejection taxonomy, and supported Lean release matrix.

Non-goals: no Z3, no proof graph, no private grind integration, no new negative theorem claim. Those follow after the native boundary is stable.

This prototype is deliberately conservative, yet it changes Leant from a separate research REPL into a practical proof-writing assistant inside the Lean editor. It also creates the exact integration surface on which typed skeletons and Z3 services can be added incrementally.

Conclusion

The reviewed repositories are unusually complementary. Lean 4 has an increasingly coherent proof-producing automation core. Leant/Djex has a disciplined structural synthesis pipeline with explicit search bounds, exact provider metadata, and mandatory Lean re-elaboration. Z3 has broad semantic search, model generation, cores, optimization, and invariant machinery. The right future direction is not to collapse these systems into one engine. It is to make their evidence compositional.

The decisive representation change is a typed, source-mapped proof graph with residual holes. It allows Leant to contribute the higher-order Curry–Howard shape, Z3 to contribute semantic choices and diagnostics, and Lean-native tactics to contribute checked theory reasoning. A common result lattice prevents search failure, solver unknown, abstract models, unsat cores, structural refutations, and kernel proofs from being confused.

The first useful step is straightforward: expose Leant as a native `lean? tactic` and verify candidates in the exact saved proof state. The next step is typed holes. Z3 should then enter as a counterexample/core/instance service, followed by EUF, arithmetic, bit-vector, and datatype reconstruction. Induction and invariants are a natural later frontier. A universal Z3 proof-object checker is optional and should be driven by measured need.

This architecture keeps the trust story simple even as the search becomes ambitious: external tools may be ingenious, incomplete, nondeterministic, or occasionally wrong; Lean accepts only an independently reconstructed proof or a certificate checked by a small proved verifier.

Capability matrix

Goal feature	Leant/Djex	Z3	Lean-native path	Recommended composition
Pure ar-rows/products/sums	Strong structural synthesis; Djinn may refute exact admitted fragment.	Propositional checking possible but loses term structure.	grind, constructors, exact.	Leant first; Lean verifies.
Higher-order hypothesis application	Strong within explicit rank-N/provider bounds.	Requires monomorphization/encoding.	Elaboration, grind instances.	Leant chooses application; Lean/Z3 fill ground arguments.
Typeclass-dependent provider	Exact bounded provider assignments; fail closed outside supported forms.	Class dictionaries should not be modeled naively.	Native typeclass synthesis.	Leant proposes provider; Lean re-synthesizes/checks instances.
Linear integer arithmetic	Structural wrapper only.	Strong models, cores, feasibility, coefficients.	omega, lia, grind.	Z3 guides; Lean arithmetic proves.
Bit vectors	Structural wrapper only.	Strong models and simplification.	bv_decide + LRAT.	Z3 diagnostics; bv_decide closes.
EUf/equalities	Can compose equality hypotheses structurally.	Strong congruence and cores.	grind, simp, congruence.	Z3 plan/core; Lean reconstructs.
Finite datatypes	Constructor/case synthesis with admitted inventories.	Models, constructor reasoning.	cases, no-confusion, grind.	Share exact family descriptor.
Arrays/maps	Limited unless providers expose lemmas.	Strong select/store and models.	rewrite/extensionality, library tactics.	Z3 plan/model; Lean leaves residual extensionality as needed.
Quantifiers	Bounded rank-N structural planning.	E-matching, MBQI, instances, unknown possible.	Explicit theorem application, grind.	Leant shape + Z3 instances + Lean checking.
Induction/recursion	Future recursor skeletons; current recursive family projection is bounded.	Invariants, Horn solving, templates.	induction, vcgen, well-founded recursion.	Leant plan + Z3 invariant + Lean VCs.
Nonlinear arithmetic	Structural wrapper only.	Models/templates; incomplete.	grobner, normalization, optional mathlib tactics.	Use Z3 heuristically; residual goals permitted.
General dependent types	Opaque transport/refusal outside admitted structure.	Not faithfully first-order in general.	Lean elaboration and user/domain tactics.	Keep opaque; use structural skeleton around exact Lean holes.

Illustrative end-to-end scenarios

B.1 Scenario A: higher-order structure plus arithmetic

1. Target contains a function hypothesis whose application will solve the goal if a natural-number side condition is proved.
2. `leant?` returns a typed application skeleton with one hole.
3. The hole classifier marks `QF_LIA`.
4. `Z3` returns a core containing two local inequalities.
5. `omega` proves the hole from those inequalities.
6. Lean reconstructs the complete graph and `TryThis` offers a readable `refine` script.

No `Z3` proof object is imported. If `Z3` is absent, the scheduler can try `omega` on all local facts; `Z3` merely reduces the search and improves explanation.

B.2 Scenario B: false finite theorem

1. Goal is a universal property of two `BitVec 8` inputs.
2. The exact bit-vector adapter negates the property and asks `Z3` for a model.
3. The model decoder obtains two concrete numerals and independently evaluates the proposition.
4. The editor shows the concrete counterexample and offers a command to insert an example regression test.
5. No proof state is changed automatically.

B.3 Scenario C: quantified algebraic theorem

1. `LibrarySuggestions` retrieves associativity, identity, and inverse theorems.
2. `Leant` determines that a quantified inverse hypothesis must be applied at a specific product term.
3. `Z3`'s finite instance loop proposes that ground instance and returns a small core.
4. Lean reconstructs the theorem application and adds it to `grind`.
5. `grind` closes the equality using congruence and rewriting.

B.4 Scenario D: loop invariant

1. Lean's VC generator produces initialization, step, and exit obligations.
2. The invariant grammar contains linear inequalities over loop indices and accumulators.

3. Z3 Optimize finds a sparse coefficient vector.
4. Lean inserts the predicate and proves all VCs using arithmetic and structural tactics.
5. If preservation fails, a validated model is converted into a concrete failing transition and the template grows.

B.5 Scenario E: proof repair

1. A renamed theorem invalidates one node in a stored proof graph.
2. The unchanged prefix reconstructs; the failed application becomes a typed hole.
3. The premise selector finds two type-compatible replacements.
4. Z3 rejects one replacement with a concrete model of its missing premise.
5. Leant composes the other replacement with an existing local hypothesis.
6. Lean accepts the new graph and renders a minimal source diff.

Source map

The following immutable links identify the principal source files and commits used in the review. They are not a complete bibliography of theorem-proving or SMT research; they are an audit trail for the repository-specific claims in this report.

Lean 4

- [leanprover/lean4@1d2dde6d64cb](#)
- [src/Lean/Elab/Tactic/Grind.lean](#)
- [src/Lean/Elab/Tactic/Grind/Main.lean](#)
- [src/Lean/Meta/Tactic/Grind/Extension.lean](#)
- [src/Lean/Meta/Tactic/Grind/Types.lean](#)
- [src/Lean/Meta/Tactic/TryThis.lean](#)
- [src/Lean/LibrarySuggestions/Basic.lean](#)
- [src/Std/Tactic/BVDecide/Reflect.lean](#)
- [commit87e7204456bf:bv_decide/grindintegration](#)
- [commitf29e9e488ea8:state-dependentterminationmeasures](#)

Leant and Djex

- [VladimirReshetnikov/Leant@dab7a1518f51](#)
- [latestfunctionalLeantintegration@0160237e4bdf](#)
- [Leant/README.md](#)
- [Leant/src/Leant/Synth/Fragment.hs](#)
- [Leant/src/Leant/Synth/Engine.hs](#)
- [Leant/src/Leant/Synth/Replay.hs](#)
- [Leant/src/Main.hs](#)
- [commit0160237e4bdf:structuralhigher-kindedssynthesissupport](#)
- [commitde4f562c77ff:six-binderDjexintegration](#)
- [VladimirReshetnikov/Djex@f50b5eac8672](#)
- [Djex/synthesis/.../Query.hs](#)
- [Djex/synthesis/.../Search.hs](#)

- `Djex/synthesis/.../Candidate.hs`
- `Djex/synthesis/.../Generated.hs`
- `Djex/synthesis/.../Selection.hs`
- `Djex/djinn/.../Djex/Djinn.hs`
- `Djex/exference/.../Djex/Exference.hs`

Z3 and comparison project

- `Z3Prover/z3@0bd679a404be`
- `z3/README.md`
- `z3/src/api/z3_api.h`
- `z3/src/api/z3_optimization.h`
- `z3/src/api/z3_fixedpoint.h`
- `z3/examples/python/proofreplay.py`
- `commitdf8d23960eda:MBPlinearityfix`
- `commit7d67dbfcfc81:quantified-realQEregression`
- `commit5bd485eb03fe:polymorphismanddependent-typeguard`
- `commit0105b220fd18:cross-platformhashdeterminism`
- `commit09aaadf96380:ASTconstructiondeterminism`
- `ufmg-smite/lean-smt@a4ed29b0b9fd`
- `lean-smt/README.md`